Segmentación de Clientes con Algoritmos de Aprendizaje Automático no Supervisado

Autor: Ernestina Campos Llano
Tutora: Dra. Andrea Rey

Universidad Nacional de Hurlingham (UNAHUR)



Índice general

Índice de Figuras **2**

Índice de Tablas **4**

1. Introducción **7**
1.1. Motivación . . . 7
1.2. Estado del Arte . . . 7

2. Metodología **8**
2.1. Conjunto de datos . . . 8
2.1.1. Limpieza de datos . . . 9
2.1.2. Análisis exploratorio: distribución geográfica . . . 9
2.2. Cálculo de RFM . . . 12
2.2.1. Escalamiento de variables . . . 13
2.3. *Clustering* . . . 15
2.3.1. K-means . . . 15
2.3.2. DBSCAN . . . 16
2.3.3. HDBSCAN . . . 16
2.4. Métricas de calidad . . . 17

3. Resultados y discusión **21**
3.1. Resultados de RFM escalado . . . 21
3.2. Resultados de K-means . . . 22
3.3. Resultados de DBSCAN . . . 24
3.4. Resultados de HDBSCAN . . . 26
3.5. Evaluación de los modelos . . . 29

4. Conclusiones **42**

Bibliografía **44**

Índice de Figuras

2.1. Top 15 países por cantidad de transacciones. 10
2.2. Top 15 países por cantidad de transacciones (excluyendo Reino Unido). 12
2.3. Distribución de las variables RFM escaladas. 14
2.4. Distribución de las variables RFM escaladas, con zoom en el eje vertical. 15
2.5. Método del codo para la selección de k. 16

3.1. Distribución de clientes según RFM. 21
3.2. Distribución de clientes según K-Means: k=4. 23
3.3. Visualización de los *clusters* ($k = 4$) en el plano Recencia-Frecuencia. 24
3.4. Visualización de los *clusters* ($k = 4$) en el plano Frecuencia-Monto. 25
3.5. Visualización con zoom de los *clusters* ($k = 4$) en el plano Frecuencia-Monto. 26
3.6. Visualización de los *clusters* ($k = 4$) en el plano Recencia-Monto. 27
3.7. Visualización con zoom de los *clusters* ($k = 4$) en el plano Recencia-Monto. 28
3.8. Distribución de clientes según *K-Means*: $k = 5$. 29
3.9. Visualización de los *clusters* ($k = 5$) en el plano Recencia-Frecuencia. 30
3.10. Visualización de los *clusters* ($k = 5$) en el plano Frecuencia-Monto. 31
3.11. Visualización con zoom de los *clusters* ($k = 5$) en el plano Frecuencia-Monto. 31
3.12. Visualización de los *clusters* ($k = 5$) en el plano Recencia-Monto. 32
3.13. Visualización con zoom de los *clusters* ($k = 5$) en el plano Recencia-Monto. 32
3.14. Distribución de clientes según DBSCAN. 33
3.15. Visualización de los *clusters* DBSCAN en el plano Recencia-Frecuencia. 33
3.16. Visualización de los *clusters* DBSCAN en el plano Frecuencia-Monto. 34
3.17. Visualización de los *clusters* DBSCAN en el plano Recencia-Monto. 34
3.18. Distribución de clientes según HDBSCAN - `min_cluster_size`=500, `min_samples`=3. . . 35
3.19. Visualización de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Recencia-Frecuencia. 35
3.20. Visualización de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Frecuencia-Monto. 36
3.21. Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Frecuencia-Monto. 36
3.22. Visualización de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plaon Recencia-Monto. 37
3.23. Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Recencia-Monto. 37
3.24. Jerarquía de *clusters* construida por HDBSCAN (`min_cluster_size`=500, `min_samples`=3). 38
3.25. Distribución de clientes según HDBSCAN - `min_cluster_size`=200, `min_samples`=1 . . 39
3.26. Visualización de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Recencia-Frecuencia. 39

3.27. Visualización de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Frecuencia-Monto. 40
3.28. Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Frecuencia-Monto. 40
3.29. Visualización de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Recencia-Monto. 41
3.30. Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Recencia-Monto. 41

Índice de Tablas

2.1. Cantidad de registros afectados por cada motivo de limpieza (*dataset* original). 9
2.2. Distribución de transacciones y facturación por país (top 10). 10
2.3. Comparación de métricas de compra entre países seleccionados. 11
2.4. Comparación entre Reino Unido vs. Resto del mundo. 13
2.5. Estadísticas descriptivas de las variables RFM. 13
2.6. Estadísticas descriptivas de las variables RFM luego del escalamiento. 14
2.7. *Grid search* de hiperparámetros para DBSCAN. 17
2.8. *Grid search* de hiperparámetros para HDBSCAN. 18

3.1. Resumen de clusters: K-means con $k = 4$. 22
3.2. Resumen de *clusters*: *K-means* con $k = 5$. 23
3.3. Resumen de *clusters*: DBSCAN (`eps`=0.2, `min_puntos`=5). 24
3.4. Resumen de *clusters*: HDBSCAN (`min_cluster_size`=500, `min_samples`=3). 26
3.5. Perfil de los *clusters* obtenidos con HDBSCAN (`min_cluster_size`=200, `min_samples`=1). 28
3.6. Comparación de métricas de evaluación entre los distintos modelos. 30

Resumen

Capítulo 1

Introducción

1.1 Motivación

En la actualidad, las plataformas de comercio electrónico (*e-commerce*) generan un volumen masivo y constante de datos sobre las transacciones de sus usuarios. Sin embargo, disponer de una gran cantidad de datos no implica necesariamente contar con información útil. El principal desafío reside en transformar esos registros transaccionales en conocimiento práctico que sirva para orientar la toma de decisiones estratégicas. En un mercado competitivo, comprender cómo se comportan los clientes, con qué frecuencia compran y cuánto gastan resulta fundamental para diseñar estrategias de retención, optimizar recursos de *marketing* y mejorar el valor a largo plazo de la base de clientes. Para abordar este análisis, es habitual recurrir a la segmentación mediante el modelo RFM (Recencia, Frecuencia y Monto) en combinación con algoritmos de aprendizaje no supervisado, *clustering*. Esta metodología permite agrupar a los clientes de forma automática según la similitud de sus patrones de compra. Sin embargo, la aplicación de estos algoritmos en conjuntos de datos reales de *e-commerce* presenta complejidades particulares. Las métricas suelen mostrar una marcada asimetría, con una gran concentración de clientes de bajo consumo y un grupo reducido de compradores mayoristas o atípicos que registran valores extremadamente altos. Teniendo esto en cuenta, los algoritmos tradicionales como *K-means*, aunque muy extendidos por su sencillez y rapidez, pueden verse afectados por la presencia de estos valores extremos o por la asunción de que los grupos tienen formas geométricas regulares. Por su parte, las alternativas basadas en densidad, como DBSCAN e HDBSCAN, proponen en teoría una mayor flexibilidad para identificar agrupaciones de formas complejas y aislar observaciones atípicas sin forzarlas a pertenecer a un clúster. La motivación de este trabajo surge de la necesidad de evaluar en la práctica el desempeño de estos tres enfoques sobre un conjunto de datos real (*Online Retail II*). El objetivo es determinar en qué medida las métricas cuantitativas de calidad matemática coinciden con la utilidad real de cada segmentación para el negocio, identificando qué algoritmo logra el equilibrio más conveniente entre precisión técnica e interpretabilidad comercial.

1.2 Estado del Arte

Capítulo 2

Metodología

2.1 Conjunto de datos

En el trabajo se utiliza el conjunto de datos *Online Retail II*, publicado en el repositorio *UCI Machine Learning Repository*, disponible en `https://archive.ics.uci.edu/dataset/502/online+retail+ii`. Contiene el registro de transacciones de una tienda minorista con sede en el Reino Unido, especializada en la venta de artículos de regalo para todo tipo de ocasiones, con una base de clientes compuesta principalmente por mayoristas.

El *dataset* abarca transacciones realizadas entre diciembre de 2009 y diciembre de 2011. Está compuesto por las siguientes variables:

- **Factura**: número de factura que identifica cada transacción.
- **Código de Artículo**: código del producto.
- **Descripción**: descripción del producto.
- **Cantidad**: cantidad de unidades vendidas por transacción.
- **Fecha de Factura**: fecha y hora de la transacción.
- **Precio**: precio unitario del producto en libras esterlinas.
- **Cliente ID**: identificador único de cliente.
- **País**: país del cliente.

El archivo original está compuesto por dos hojas de cálculo, correspondientes a los períodos 2009-2010 y 2010-2011 respectivamente. Se generó un módulo en Python que lee ambas hojas y las concatena en un único DataFrame antes de generar una copia local en formato `.csv`.

Para la carga de los datos se desarrolló un procedimiento que lee el archivo original en formato Excel (`.xlsx`) y lo convierte a un archivo `.csv` que se guarda como una copia local. En las ejecuciones siguientes, el proceso verifica si ya existe el archivo `.csv` y, en caso afirmativo, lo utiliza directamente en lugar de volver a leer el archivo original. Este mecanismo reduce significativamente el tiempo de carga, dado que la lectura de archivos Excel de gran tamaño es considerablemente más lenta que la de archivos `.csv` de igual contenido.

El *dataset* original está compuesto por 1.067.371 registros y 7 características. Las fechas de las transacciones abarcan desde el 1 de diciembre de 2009 hasta el 9 de diciembre de 2011. Se identificaron 5.942 clientes únicos y transacciones provenientes de 43 países distintos, con predominio de Reino Unido tal como lo muestra la Figura 2.1.

Un análisis inicial de calidad de datos reveló los siguientes puntos a resolver en la etapa de limpieza:

- **Valores faltantes**: 243.007 registros (22,8 %) sin `Cliente ID` y 4.382 registros (0,41 %) sin `Descripción`.
- **Duplicados**: 34.335 registros duplicados (3,22 %).
- **Cancelaciones**: 19.494 transacciones (1,83 %) corresponden a cancelaciones, identificables porque su número de factura (`Factura`) empieza con la letra "C".
- **Valores negativos**: el atributo `Cantidad` presenta un mínimo de -80.995 unidades, y la columna `Precio` un mínimo de -$53.594,36.

2.1.1 Limpieza de datos

A partir del análisis inicial descrito en la sección anterior, se procedió a limpiar el *dataset*, aplicando los siguientes filtros:

1. **Eliminación de registros sin** `Customer ID`: dado que las transacciones sin identificador de cliente no pueden asociarse a ningún segmento y se descartan.
2. **Eliminación de cancelaciones**: se eliminan las transacciones cuyo número de `Factura` comienza con la letra "C", correspondientes a cancelaciones.
3. **Eliminación de valores inválidos**: se conservan únicamente los registros con `Cantidad` y `Precio` positivos, descartando así cantidades o precios negativos que no representan compras válidas.
4. **Eliminación de duplicados**: se remueven los registros completamente duplicadas.

Para entender mejor el impacto de cada filtro, podemos observar la Tabla 2.1 que cuenta por separado cuántos registros cumplen cada uno de estos motivos sobre el *dataset* original. Hay que tener en cuenta que estos motivos no son excluyentes: un mismo registro puede cumplir varios a la vez, por ejemplo, no tener `Cliente ID` y ser también una cancelación.

Tabla 2.1: Cantidad de registros afectados por cada motivo de limpieza (*dataset* original).

Motivo	Registros afectados
Sin `Cliente ID`	243.007
Cancelaciones	19.494
`Cantidad` con valores negativos	22.950
`Precio` con valores negativos	6.207

Luego de esta limpieza, el *dataset* pasó de 1.067.371 a 779.425 registros, es decir que se eliminó un 27,0 % de los registros originales. La cantidad de clientes únicos casi no cambió, pasando de 5.942 a 5.878, lo que muestra que la mayoría de los clientes tiene al menos una transacción válida después de la limpieza.

2.1.2 Análisis exploratorio: distribución geográfica

Como parte del análisis exploratorio inicial, se analizó cómo se distribuían las transacciones según el país de origen del cliente, con el objetivo de ver si había diferencias de comportamiento entre países que fueran relevantes.

El *dataset* contiene registros de 43 países pero, como se puede observar en la Tabla 2.2, hay una concentración muy marcada en el Reino Unido. El mismo concentra el 89,86 % de los registros del *dataset* y el 82,82 % de la facturación total.

Tabla 2.2: Distribución de transacciones y facturación por país (top 10).

País	Importe total	% Importe	Registros	% Registros
Reino Unido	14.389.230	82,82	700.388	89,86
Irlanda	616.570	3,55	15.565	2,00
Países Bajos	554.038	3,19	5.085	0,65
Alemania	425.020	2,45	16.432	2,11
Francia	348.769	2,01	13.511	1,73
Australia	169.284	0,97	1.789	0,23
España	108.333	0,62	3.662	0,47
Suiza	100.062	0,58	3.005	0,39
Suecia	91.516	0,53	1.317	0,17
Dinamarca	68.581	0,39	778	0,10

La Figura 2.1 muestra los 15 países con más transacciones. Como Reino Unido domina la escala del gráfico, se agrega la Figura 2.2, sin Reino Unido, para poder ver mejor cómo se distribuye el resto de los países.

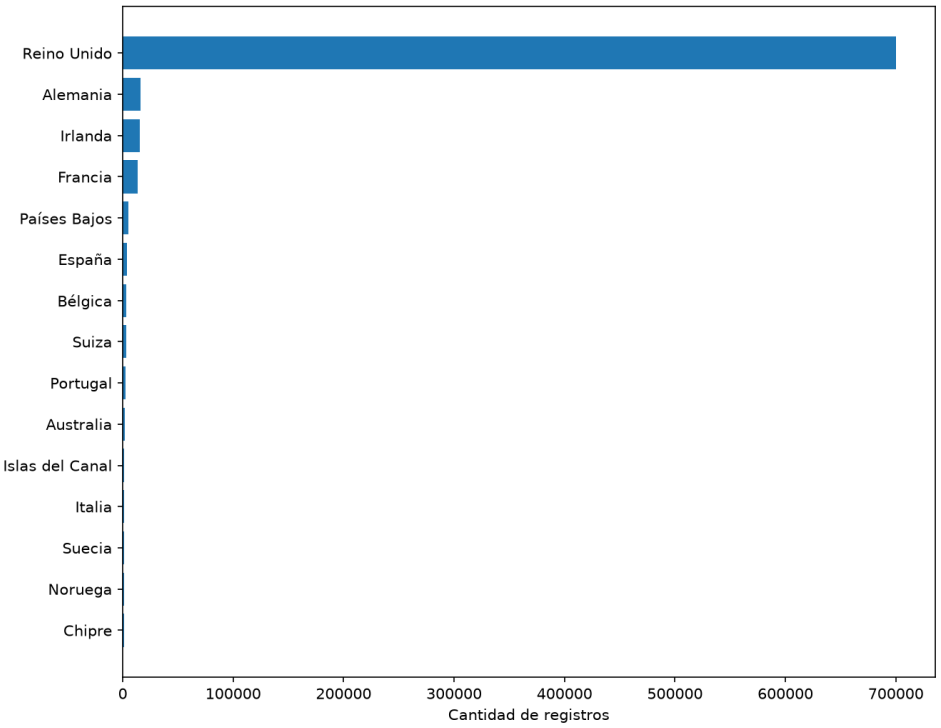


Figura 2.1: Top 15 países por cantidad de transacciones.

Para ver si el comportamiento de compra cambia según el país, se compararon algunos de distintas regiones, tanto en cantidad de transacciones como en el valor de las mismas. En la Tabla 2.3 se presentan estas métricas para una selección de países de distinto tamaño y ubicación geográfica.

Las comparaciones muestran un patrón que se repite en la mayoría de los pares de países analizados: los países con menos clientes y transacciones tienen un ticket promedio por factura más alto que aquellos que tienen más cantidad de clientes y transacciones. El caso más marcado es el de Países Bajos frente a Alemania: con menos de un tercio de los registros (5.085 frente a 16.432), Países Bajos tiene un ticket promedio 4,5 veces mayor (2.429,99 contra 538,68). En el caso de Francia y Alemania, ambos países

Tabla 2.3: Comparación de métricas de compra entre países seleccionados.

	Reino Unido	Lituania	Francia	Alemania	Emiratos Árabes	Portugal	Países Bajos	Japón	Suiza
Registros	700.388	154	13.511	16.432	383	2.356	5.085	468	3.005
Facturas únicas	33.541	6	614	789	11	93	228	33	90
Clientes únicos	5.350	1	95	107	4	24	22	10	22
Importe promedio (por línea)	20,54	31,77	25,81	25,87	24,03	23,58	108,96	91,93	33,30
Cantidad promedio (por línea)	12,18	14,97	20,01	13,70	15,25	11,64	75,49	67,61	17,38
Precio promedio	3,07	2,56	4,29	3,59	3,64	5,10	2,69	2,01	3,82
Ticket promedio por factura	429,00	815,45	568,03	538,68	836,61	597,36	2.429,99	1.303,75	1.111,80
Ítems promedio por factura	254,39	384,33	440,21	285,37	530,82	294,87	1.683,68	958,88	580,30

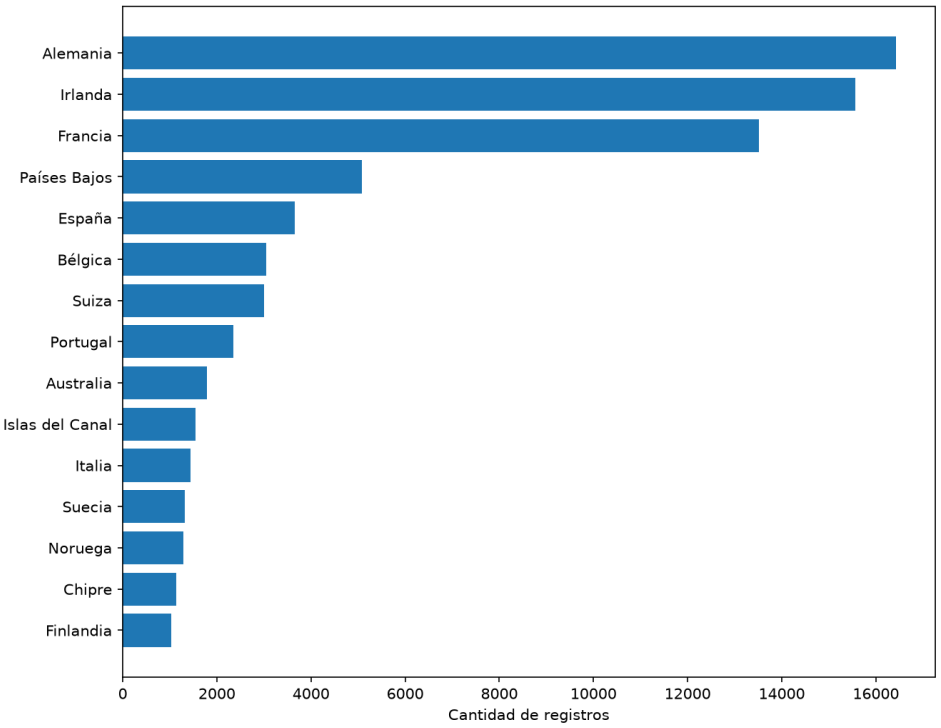


Figura 2.2: Top 15 países por cantidad de transacciones (excluyendo Reino Unido).

tienen una cantidad de registros y facturas similares entre sí, es consistente con el patrón general, sus valores de ticket promedio e importe promedio por ítem también son parecidos entre sí, lo que refuerza la idea de que la diferencia de ticket promedio está más asociada al volumen relativo de cada país que a otro factor.

La comparación entre Reino Unido y el resto del mundo, presentada en la Tabla 2.4, resulta particularmente interesante. Reino Unido concentra el 89,86 % de las transacciones, pero el resto del mundo tiene un ticket promedio por factura de 870,94, más del doble que Reino Unido (429,00), y también más ítems promedio por factura (578,05 frente a 254,39). Esto sugiere que los clientes fuera de Reino Unido tienen, en promedio, un perfil de compra más parecido al de un mayorista o distribuidor, mientras que dentro de Reino Unido el comportamiento se parece más al de un consumidor minorista.

2.2 Cálculo de RFM

A partir del *dataset* limpio y filtrado solamente con los registos pertenecientes a Reino Unido, se calcularon para cada cliente las tres variables que componen el mdoelo RFM:

- **Recencia**: cantidad de días transcurridos entre la fecha de la última compra del cliente y una fecha de referencia fija, correspondiente al día posterior a la última transacción registrada en el *dataset*.
- **Frecuencia**: cantidad de facturas únicas asociadas al cliente, es decir, la cantidad de compras distintas que realizó.
- **Monto**: suma total del importe de todas las transacciones del cliente.

Este cálculo se realizó unicamente sobre los clientes de Reino Unido, dado que, como se discutió en la sección anterior, este segmento concentra la enorme mayoría de los registros y presenta un comportamiento de compra distinto al del resto del mundo, más parecido al de un consumidor minorista. El

Tabla 2.4: Comparación entre Reino Unido vs. Resto del mundo.

	Reino Unido	Resto del mundo
Registros	700.388	79.037
Facturas únicas	33.541	3.428
Clientes únicos	5.350	529
Importe promedio	20,54	37,77
Cantidad promedio	12,18	25,07
Precio promedio	3,07	4,57
Importe promedio por factura	429,00	870,94
Ítems promedio por factura	254,39	578,05

resultado es un conjunto de 5350 clientes únicos, cada uno caracterizado por sus tres métricas RFM. La Tabla 2.5 resume la distribución estadística de estas variables.

Tabla 2.5: Estadísticas descriptivas de las variables RFM.

	Recencia	Frecuencia	Monto
Media	203,00	6,27	2.689,58
Desvío estándar	209,96	12,00	11.706,53
Mínimo	1	1	2,95
Primer Cuartil	26	1	330,20
Mediana	98,5	3	829,44
Tercer Cuartil	382	7	2.156,25
Máximo	739	336	580.987,04

Las tres variables tienen una distribución bastante asimétrica: hay una diferencia grande entre la mediana, la media y el máximo. Esto se ve más que nada en Monto, donde la mediana es de $829,44, la media sube a $2689,58 y el máximo llega a $580.987,04. Esa diferencia tan grande muestra que hay clientes que compran mucho más que el resto.

2.2.1 Escalamiento de variables

Dado que las tres variables RFM se encuentran en escalas muy distintas entre sí (días, cantidad de compras y monto en dólares), respectivamente fue necesario estandarizarlas antes de aplicar cualquier algoritmo de *clustering*. Si no se hacía esto, el Monto iba a dominar el cálculo de distancias entre clientes, ya que tiene una magnitud mucho mayor que la Recencia o la Frecuencia, y eso iba a distorsionar los resultados. Para esto se aplicó una estandarización de tipo *z-score*, que a cada variable le resta su media y la divide por su desvío estándar, para que las tres variables de interes queden centradas en cero y con varianza 1.

Tabla 2.6: Estadísticas descriptivas de las variables RFM luego del escalamiento.

	Recencia	Frecuencia	Monto
Media	≈ 0	≈ 0	≈ 0
Desvío estándar	1,00	1,00	1,00
Mínimo	-0,96	-0,44	-0,23
Primer Cuartil	-0,84	-0,44	-0,20
Mediana	-0,50	-0,27	-0,16
Tercer Cuartil	0,85	0,06	-0,05
Máximo	2,55	27,49	49,40

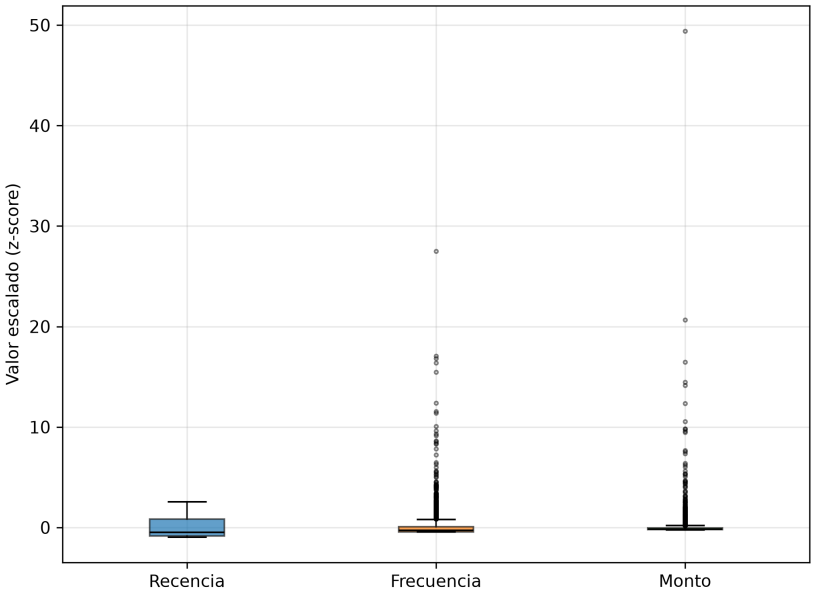


Figura 2.3: Distribución de las variables RFM escaladas.

En la Tabla 2.6 se puede observar que, aunque el escalamiento logra que las tres variables queden en un rango comparable en su parte central (media 0, desvío 1), los valores máximos de Frecuencia y Monto siguen siendo extremos (27,49 y 49,40 respectivamente). Esto es esperable, porque la estandarización solo cambia la escala y no la forma de la distribución: la asimetría de estas variables es propia de los datos y responde a un pequeño grupo de clientes con un comportamiento de compra muy por encima del resto. En el *boxplot* de la Figura 2.3 se ve lo mismo: la Recencia no presenta *outliers*, mientras que la Frecuencia y el Monto tienen muchos puntos que se alejan de la caja, sobre todo ese caso en el Monto que llega casi a 50. Como esos valores extremos aplastan las cajas contra el cero, la Figura 2.4 muestra el mismo *boxplot* con el eje vertical limitado a valores entre -1 y 10, para poder ver mejor las cajas de Frecuencia y Monto. Esta versión deja fuera de la vista a los *outliers* más alejados, que igual se tienen en cuenta en el análisis.

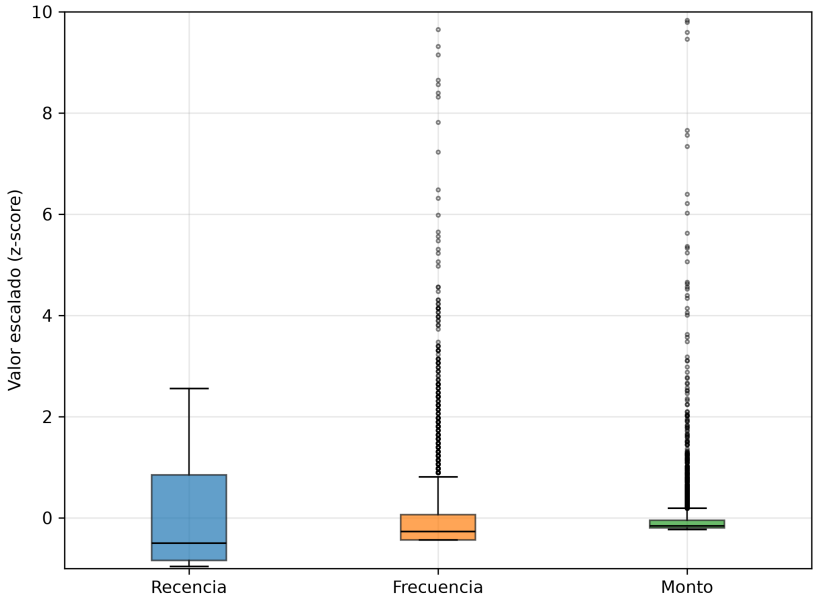


Figura 2.4: Distribución de las variables RFM escaladas, con zoom en el eje vertical.

2.3 *Clustering*

El objetivo general de este trabajo es comparar el desempeño de *k-medias* y DBSCAN para la segmentación de clientes en un contexto de *e-commerce*, evaluando métricas internas e interpretando las características de cada uno de los segmentos. Además de estos dos algoritmos, se agregó HDBSCAN porque resuelve una limitación de DBSCAN: este define los vecindarios con un único radio (`eps`) para todo el *dataset*, y por eso le cuesta cuando hay zonas con densidades distintas, mientras que HDBSCAN no depende de ese radio fijo y se adapta mejor a esa variación de densidad.

En las siguientes subsecciones se describe cómo se aplicó cada uno de los tres algoritmos sobre las variables RFM escaladas, y qué criterios se utilizaron para seleccionar sus respectivos parámetros. En todas ellas, un punto es un registro del dataset, es decir, un cliente descrito por sus variables numéricas (Recencia, Frecuencia y Monto).

2.3.1 K-means

El primer algoritmo aplicado fue *K-means*, un método de *clustering* particional que agrupa las observaciones en k clusters, donde k es justamente el valor que da nombre al método, buscando minimizar la distancia entre cada punto y el centroide de su *cluster*. A diferencia de DBSCAN y HDBSCAN, *K-means* requiere que se especifique de antemano la cantidad de *clusters* a formar, por lo que el primer paso fue determinar un valor adecuado de k. Para esto se utilizó el método del codo, que consiste en ejecutar el algoritmo con distintos valores de k y registrar, en cada caso, la inercia resultante (la suma de las distancias al cuadrado entre cada punto y su centroide). A medida que k aumenta, la inercia disminuye de forma natural, pero llega un punto en el que agregar *clusters* adicionales deja de aportar una mejora significativa. Ese punto de quiebre, visualizado como un "codo" en el gráfico, se toma como una buena referencia para elegir k.

La Figura 2.5 muestra la inercia obtenida para valores de k entre 2 y 10. Se ve una caída pronunciada hasta $k = 4$ o $k = 5$ y, a partir de ahi la curva se va aplanando considerablemente. Tomando esto como

referencia, se probó el algoritmo tanto con $k = 4$ como con $k = 5$, para comparar ambas segmentaciones y evaluar cuál resulta más interpretable y proporciona una mejor separación de grupos según las métricas internas Silhouette, Davies-Bouldin y Calinski-Harabasz, que se detallan en la Sección 2.4.

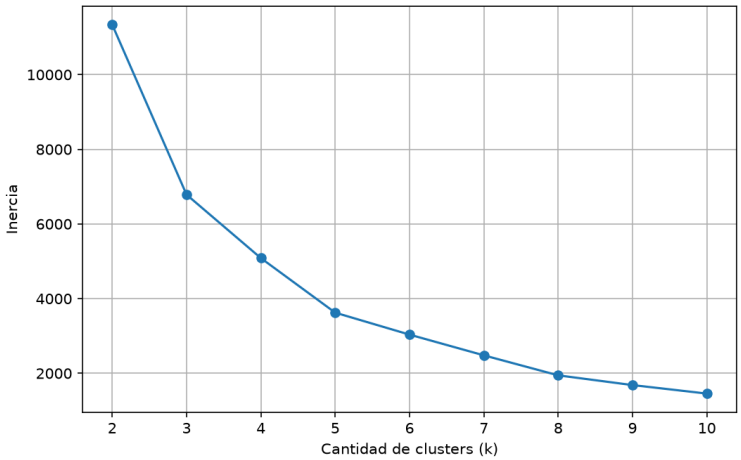


Figura 2.5: Método del codo para la selección de k.

2.3.2 DBSCAN

El segundo algoritmo aplicado fue DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*), un método de *clustering* basado en densidad que agrupa los puntos que están suficientemente cerca entre sí, y marca como ruido a los que quedan aislados. A diferencia de *K-means*, DBSCAN no necesita que se le indique de antemano la cantidad de clusters a formar; en cambio, requiere dos parámetros: `eps`, la distancia máxima entre dos puntos para que se consideren vecinos, y `min_puntos`, la cantidad mínima de vecinos que necesita un punto para ser considerado parte de un *cluster*. Para elegir estos parámetros se hizo una búsqueda combinando distintos valores de `eps` y `min_puntos`, y para cada combinación se registró la cantidad de *clusters* formados y la cantidad (y porcentaje) de puntos clasificados como ruido. Las primeras corridas, con valores de `eps` más altos, agrupaban a casi todos los clientes en un único *cluster*, mientras que valores muy bajos de `eps` fragmentaban el *dataset* en decenas de *clusters* de apenas 3 o 4 clientes cada uno. Ninguno de estos dos extremos resulta útil para una segmentación interpretable. La Tabla 2.7 muestra los resultados de probar distintas combinaciones de `eps` y `min_puntos` donde se ve un patrón bastante claro: a medida que `eps` crece, el algoritmo tiende a juntar todo en un solo *cluster*, y el porcentaje de ruido baja cada vez más. Con valores de `eps` chicos pasa lo contrario, aparecen más *clusters*, pero también sube el ruido. Se terminó eligiendo `eps`=0,2 y `min_puntos`=5, que da 4 *clusters* con un 3,40 % de ruido. Aunque hay combinaciones con menos ruido, la mayoría de esas terminan agrupando a casi todos los clientes en un único *cluster*, lo cual no sirve para el objetivo de armar una segmentación.

2.3.3 HDBSCAN

El tercer algoritmo que se aplicó fue HDBSCAN (*Hierarchical Density-Based Spatial Clustering of Applications with Noise*). A diferencia de DBSCAN, no hace falta fijar un valor de `eps` de antemano debido a que esta técnica construye una jerarquía de *clusters* a distintos niveles de densidad y selecciona automáticamente los *clusters* más estables de esa jerarquía. El parámetro principal es `min_cluster_size`, el tamaño mínimo que tiene que tener un grupo para considerarse un *cluster*; también se puede ajustar `min_samples`, que regula qué tan estricto es el algoritmo a la hora de de-

eps	min_puntos	*Clusters*	Puntos de ruido	% ruido
0,2	3	7	154	2,88
0,2	5	4	182	3,40
0,2	10	2	241	4,50
0,2	15	1	299	5,59
0,3	3	8	87	1,63
0,3	5	1	125	2,34
0,3	10	1	149	2,79
0,3	15	1	163	3,05
0,4	3	5	59	1,10
0,4	5	2	81	1,51
0,4	10	1	112	2,09
0,4	15	1	122	2,28
0,7	3	4	39	0,73
0,7	5	2	45	0,84
0,7	10	1	59	1,10
0,7	15	1	69	1,29
1,0	3	3	38	0,71
1,0	5	1	44	0,82
1,0	10	1	48	0,90
1,0	15	1	50	0,93
1,5	3	4	21	0,39
1,5	5	1	34	0,64
1,5	10	1	37	0,69
1,5	15	1	37	0,69

Tabla 2.7: *Grid search* de hiperparámetros para DBSCAN.

cidir qué es ruido. Al igual que con DBSCAN, se hizo una búsqueda combinando distintos valores de `min_cluster_size` y `min_samples`. La Tabla 2.8 muestra los resultados de probar distintas combinaciones de `min_cluster_size` y `min_samples`. Con valores bajos de `min_cluster_size` (entre 10 y 100), el algoritmo formaba muchos *clusters* chicos, entre 50 y 115, algo poco útil para una segmentación interpretable. A medida que se subió el valor para `min_cluster_size`, la cantidad de *clusters* bajó de forma consistente, hasta llegar a valores más manejables (4 a 7 *clusters*) con `min_cluster_size` entre 300 y 500. Un aspecto que se mantuvo constante en toda la búsqueda fue el porcentaje de ruido, que se movió siempre entre 20 % y 39 % sin importar mucho la combinación de parámetros elegida. En un principio se había elegido `min_cluster_size`=500 y `min_samples`=3, buscando una cantidad de *clusters* parecida a la que se usó en *K-means*. Pero después se probó también la combinación con menos ruido de toda la grilla de busqueda, `min_cluster_size`=200 y `min_samples`=1, para ver qué tan buena era esa alternativa.

2.4 Métricas de calidad

Para evaluar la calidad de los *clusters* obtenidos con cada algoritmo se utilizaron tres métricas de evaluación. A diferencia de un problema de clasificación, en *clustering* no existe una respuesta correcta conocida de antemano para cada cliente, por lo que estas métricas evalúan directamente la estructura de los *clusters* (qué tan compactos son y qué tan separados están entre sí) en lugar de comparar contra un

`min_cluster_size`	`min_samples`	Cantidad de *Clusters*	Puntos de ruido	% ruido
100	1	15	1.274	23,81
100	2	14	1.406	26,28
100	3	14	1.470	27,48
100	5	13	1.666	31,14
100	10	10	1.972	36,86
150	1	10	1.195	22,34
150	2	10	1.223	22,86
150	3	10	1.282	23,96
150	5	10	1.401	26,19
150	10	9	1.846	34,50
200	1	9	1.173	21,93
200	2	9	1.197	22,37
200	3	8	1.449	27,08
200	5	9	1.576	29,46
200	10	8	2.040	38,13
300	1	7	1.668	31,18
300	2	7	1.686	31,51
300	3	7	1.718	32,11
300	5	7	1.793	33,51
300	10	6	1.812	33,87
500	1	5	1.704	31,85
500	2	5	1.723	32,21
500	3	5	1.760	32,90
500	5	5	1.855	34,67
500	10	4	2.094	39,14

Tabla 2.8: *Grid search* de hiperparámetros para HDBSCAN.

resultado esperado, como en las técnicas de aprendizaje automático supervisado.

El coeficiente de Silhouette mide qué tan bien separado está cada punto de los demás *clusters* en relación a su propio *cluster*. La idea de fondo es simple: un punto está bien agrupado si está cerca de los demás puntos de su *cluster* y, al mismo tiempo, lejos de los puntos del *cluster* más cercano. Para cuantificar esto, para cada observación se calculan dos distancias promedio: la distancia promedio a los puntos de su propio *cluster*, y la distancia promedio a los puntos del *cluster* vecino más cercano (es decir, el segundo *cluster* más parecido a esa observación). Después se calcula la diferencia entre ambas distancias, normalizada para quedar en un rango de -1 a 1:

$$s(i) = \frac{b(i) - a(i)}{\text{máx}\{a(i), b(i)\}} \tag{2.1}$$

donde $a(i)$ es la distancia promedio entre el punto i y el resto de los puntos de su mismo *cluster*, y $b(i)$ es la distancia promedio entre el punto i y los puntos del *cluster* vecino más cercano.

El valor de $s(i)$ se puede interpretar de la siguiente manera: si $b(i)$ es mucho mayor que $a(i)$, quiere decir que el punto i está mucho más cerca de su propio *cluster* que del *cluster* vecino, y $s(i)$ se acerca a 1, lo cual indica una buena asignación. Si $a(i)$ y $b(i)$ son parecidos, $s(i)$ se acerca a 0, lo que sugiere que el punto está en el límite entre dos *clusters*. Y si $a(i)$ termina siendo mayor que $b(i)$, es decir que el punto está en realidad más cerca del *cluster* vecino que del suyo propio, $s(i)$ toma valores negativos, lo que sugiere que probablemente fue asignado al *cluster* equivocado.

El coeficiente de Silhouette global de un modelo se obtiene promediando $s(i)$ sobre todas las observaciones, y sirve como una medida general de qué tan bien definida está la segmentación en su conjunto.

El índice de Davies-Bouldin DB evalúa la relación entre la dispersión dentro de cada *cluster* y la distancia entre los centros de los distintos *clusters*. La lógica detrás de esta métrica es que un buen *cluster* debería ser compacto (los puntos dentro de él no deberían estar muy dispersos) y, al mismo tiempo, debería estar bien alejado de los demás *clusters*. Para capturar esta característica, para cada *cluster* se busca el peor caso posible: el *cluster* vecino con el cual la relación entre dispersión y distancia sea más desfavorable, es decir, aquel que combine alta dispersión con poca distancia entre centroides:

$$DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left\{ \frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right\} \tag{2.2}$$

donde k es la cantidad de *clusters*, σ_i es la dispersión promedio dentro del *cluster* i (qué tan lejos están en promedio los puntos del *cluster* respecto a su centroide), y $d(c_i, c_j)$ es la distancia entre los centroides de los *clusters* i y j para $i, j = 1, 2, \ldots, k$.

Para cada *cluster* i, el término $\frac{\sigma_i + \sigma_j}{d(c_i, c_j)}$ se calcula contra todos los demás *clusters* j, y se queda con el valor máximo, es decir, con la peor comparación posible para ese *cluster*. Esto tiene sentido porque a un *cluster* le alcanza con estar mal separado de uno solo de sus vecinos para que se considere una segmentación ineficiente, aunque esté bien separado del resto. Después, el índice de Davies-Bouldin final se obtiene promediando ese peor caso entre todos los *clusters*.

A diferencia del Silhouette, en este caso valores más bajos indican mejor separación, ya que un buen resultado implica *clusters* compactos (dispersión baja) y bien distanciados entre sí, lo cual hace que el cociente completo se achique.

El índice de Calinski-Harabasz CH, también conocido como criterio de razón de varianza, compara la dispersión entre *clusters* con la dispersión dentro de cada *cluster*. La idea es parecida a la de un análisis de varianza: si los *clusters* están bien definidos, los puntos deberían variar poco respecto al centro de su propio *cluster* (dispersión dentro del *cluster*), pero los centros de los distintos *clusters* deberían estar bastante alejados entre sí (dispersión entre *clusters*). El índice se calcula como:

$$CH = \frac{\operatorname{tr}(B_k)}{\operatorname{tr}(W_k)} \times \frac{n - k}{k - 1} \tag{2.3}$$

donde n es la cantidad total de observaciones, k la cantidad de *clusters*, B_k la matriz de dispersión entre *clusters* y W_k la matriz de dispersión dentro de los *clusters*.

La matriz B_k mide qué tan separados están los centroides de cada *cluster* respecto al centroide general de todos los datos, ponderado por la cantidad de puntos en cada *cluster*; su traza resume esa dispersión entre *clusters* en un solo número. La matriz W_k, por su parte, mide qué tan dispersos están los puntos dentro de cada *cluster* respecto a su propio centroide, y su traza resume esa dispersión interna. El cociente entre las tazas da entonces una idea directa de qué tan grande es la separación entre *clusters* en relación a la compactación dentro de cada uno.

El factor $\frac{n-k}{k-1}$ es un ajuste que tiene en cuenta la cantidad de observaciones y de *clusters*, y sirve para que el índice no favorezca automáticamente a soluciones con más o menos *clusters* solo por la forma en que se calculan las matrices de dispersión (B_k y W_k), permitiendo comparar configuraciones con distinta cantidad de *clusters* de forma más justa.

En este caso valores más altos son mejores, porque reflejan una situación en la que la dispersión entre *clusters* es grande en relación a la dispersión dentro de cada *cluster*, es decir, *clusters* densos y bien diferenciados entre sí.

Es importante aclarar que estas tres métricas se calcularon únicamente sobre el modelo final de cada algoritmo, y no se usaron durante la búsqueda de hiperparámetros de DBSCAN y HDBSCAN. Esto se debe a que tanto DBSCAN como HDBSCAN pueden dejar puntos sin asignar a ningún *cluster*,

marcándolos como ruido (*cluster* -1), y las tres métricas mencionadas no fueron diseñadas para manejar este tipo de puntos.

En el caso del Silhouette y del índice de Calinski-Harabasz, el cálculo requiere comparar cada punto contra un *cluster* de referencia; si los puntos de ruido se incluyen como si fueran un *cluster* más, se termina midiendo la separación de un grupo que en realidad no representa ningún patrón real, sino simplemente todo lo que el algoritmo no pudo agrupar. Esto puede distorsionar el resultado tanto para bien como para mal, dependiendo de cuántos puntos caigan en esa categoría y qué tan dispersos estén. El índice de Davies-Bouldin presenta un problema similar, ya que también depende de calcular centroides y dispersiones por *cluster*, algo que no tiene mucho sentido para un conjunto de puntos que fueron agrupados justamente por no pertenecer a ningún *cluster*.

Por este motivo, usar estas métricas durante la etapa de búsqueda de hiperparámetros hubiera sido poco confiable, ya que un modelo con mucho ruido podría obtener un buen puntaje simplemente porque el ruido no está siendo evaluado correctamente, y no porque los *clusters* formados sean realmente buenos. Por eso se optó por evaluar los hiperparámetros de DBSCAN y HDBSCAN en base a criterios más interpretables, como la cantidad de *clusters* obtenidos y el porcentaje de puntos clasificados como ruido, y reservar las métricas de Silhouette, Davies-Bouldin y Calinski-Harabasz para la comparación final entre los tres modelos.

Capítulo 3

Resultados y discusión

3.1 Resultados de RFM escalado

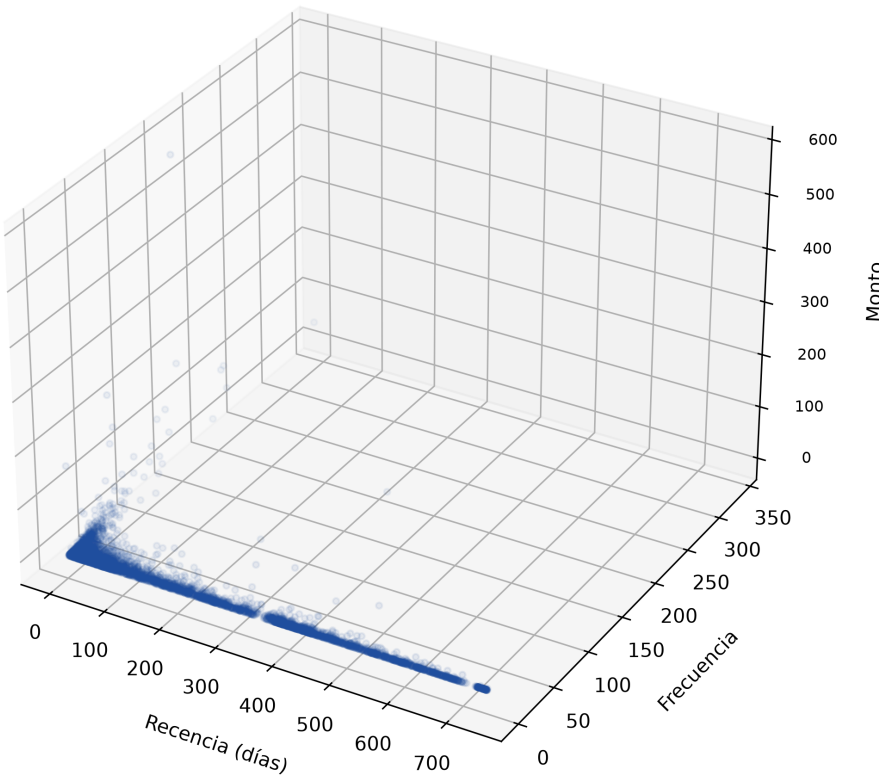


Figura 3.1: Distribución de clientes según RFM.

En la Figura 3.1 se puede apreciar la distribución de los clientes según el modelo RFM escalado. La gran mayoría de los clientes se concentra cerca del origen, con valores bajos de Frecuencia y Monto, mientras que aparecen algunos pocos casos bien alejados del resto en esas dos variables, que son los mismos que ya se habían visto como *outliers* en el *boxplot* de la Sección 2.2.1. En cuanto a la Recencia, los clientes se reparten de forma más pareja a lo largo de todo el eje, sin ese mismo patrón de concentración. Esta

forma de la nube de puntos, una masa grande y compacta de clientes parecidos entre sí, es justamente, como se va a ver más adelante, lo que complica a algoritmos como DBSCAN a la hora de separar bien los grupos. Por eso, como se va a analizar en la siguiente sección, conviene también mirar la distribución de a pares de variables.

3.2 Resultados de K-means

Se ejecutó K-means con $k = 4$ y con $k = 5$, tal como se planteó en la Sección 2.3.1. Para cada *cluster* se calcularon los promedios de Recencia, Frecuencia y Monto, para poder interpretar qué tipo de cliente representa cada grupo.

K-means con $k = 4$

Tabla 3.1: Resumen de clusters: K-means con $k = 4$.

Cluster	Clientes	Recencia promedio	Frecuencia promedio	Monto promedio
0	1.842	462,79	2,19	683,60
1	34	26,26	104,62	80.479,43
2	3.473	67,00	7,43	2.825,44
3	1	1,00	145,00	580.987,04

La Tabla 3.1 y la Figura 3.2 muestran que el *cluster* 2 es el más numeroso, con 3.473 clientes: compran cada dos meses aproximadamente y gastan en promedio $2.825,44, un perfil de cliente activo y regular. El *cluster* 0, con 1.842 clientes, agrupa a quienes hace mucho que no compran (462 días en promedio) y con un gasto bajo, lo que sugiere clientes inactivos o perdidos. El *cluster* 1, más chico (34 clientes), es el que más compra y más gasta: más de 100 facturas y cerca de $80.479 en promedio, muy por encima del resto. Por último, el *cluster* 3 queda formado por un único cliente, con un monto total de $580.987,04 y 145 facturas, muy por encima de cualquier otro cliente de la base. Este resultado es consistente con lo discutido en la Sección 2.2.1, donde ya se había identificado la presencia de un cliente con valores de compra extremos respecto al resto.

Para complementar la vista en tres dimensiones, también se armaron gráficos de a pares de variables, que permiten ver mejor cómo quedan separados los grupos. Estos gráficos son solo una forma de visualizar el resultado: los *clusters* se calcularon con las tres variables a la vez, y cada figura muestra la proyección de esos mismos grupos sobre dos de ellas. En la Figura 3.3 se aprecia bastante bien la diferencia entre los *clusters*: el grupo con mayor recencia y baja frecuencia queda claramente aparte del resto, mientras que el cluster más numeroso se concentra en valores bajos de recencia. Al mirar los pares que se muestran en Figuras 3.4 y 3.6 que incluyen el Monto, aparece de nuevo el mismo problema que se vio en el escalamiento: hay un cliente con un monto tan alto que el resto de los puntos queda apretado contra el eje, y no se distinguen bien los *clusters*. Por eso se armó una segunda versión de cada gráfico, dejando fuera de la vista ese valor extremo, sin sacarlo del análisis, solo de la figura, para poder ver mejor cómo se separan los demás *clusters*. Se puede ver en las versiones con zoom de las Figuras 3.5 y 3.7 que casi no aparecen los *clusters* de mayor monto y frecuencia, ya que son justamente los que quedan afuera del recorte pero no porque hayan desaparecido, sino porque son los valores más altos que se dejaron fuera a propósito.

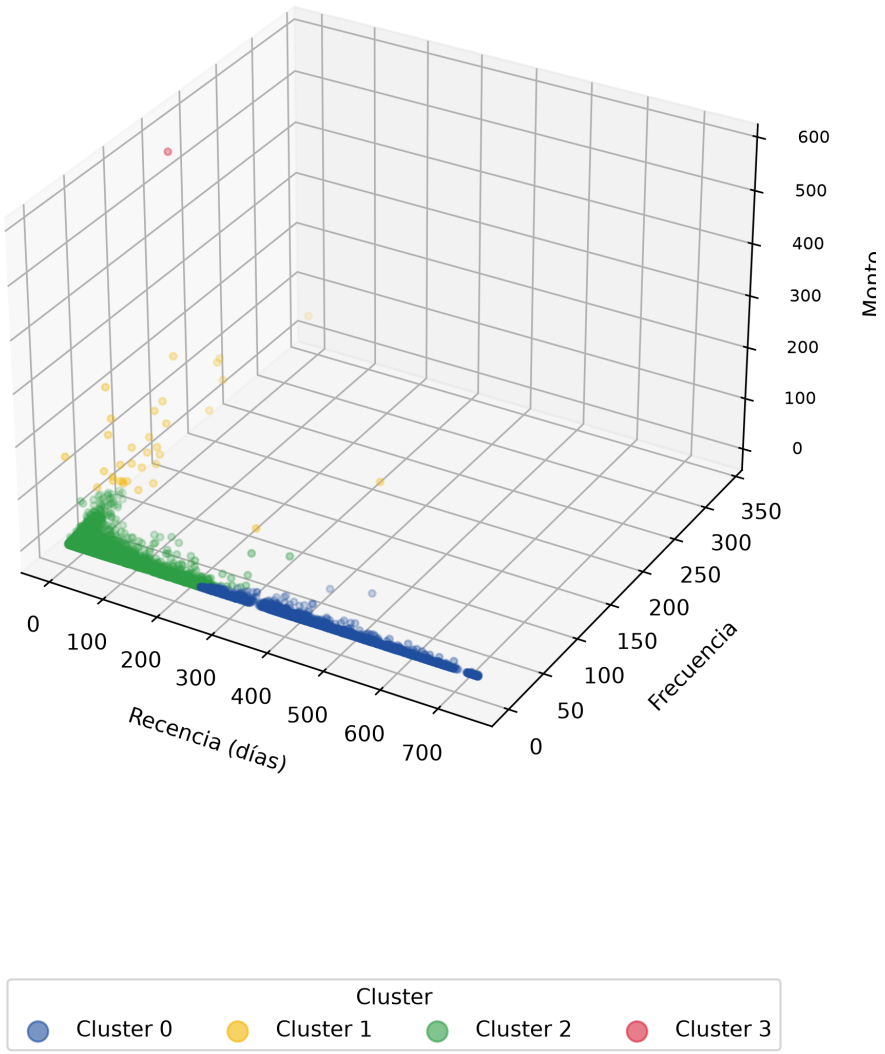


Figura 3.2: Distribución de clientes según K-Means: k=4.

***K-means* con $k = 5$**

Tabla 3.2: Resumen de *clusters*: *K-means* con $k = 5$.

Cluster	Clientes	Recencia promedio	Frecuencia promedio	Monto promedio
0	3.240	74,82	5,36	1.794,78
1	21	23,86	125,62	103.602,85
2	1.773	470,49	2,17	678,72
3	315	28,48	30,31	14.647,99
4	1	1,00	145,00	580.987,04

Por otro lado, podemos ver en la Tabla 3.2 y en la Figura 3.8 que con $k = 5$, los *clusters* de clientes inactivos (*cluster* 2) y de clientes regulares (*cluster* 0) se mantienen prácticamente iguales a los obtenidos con $k = 4$. La diferencia más importante está en el *cluster* de alto valor que teníamos con $k = 4$ el cual

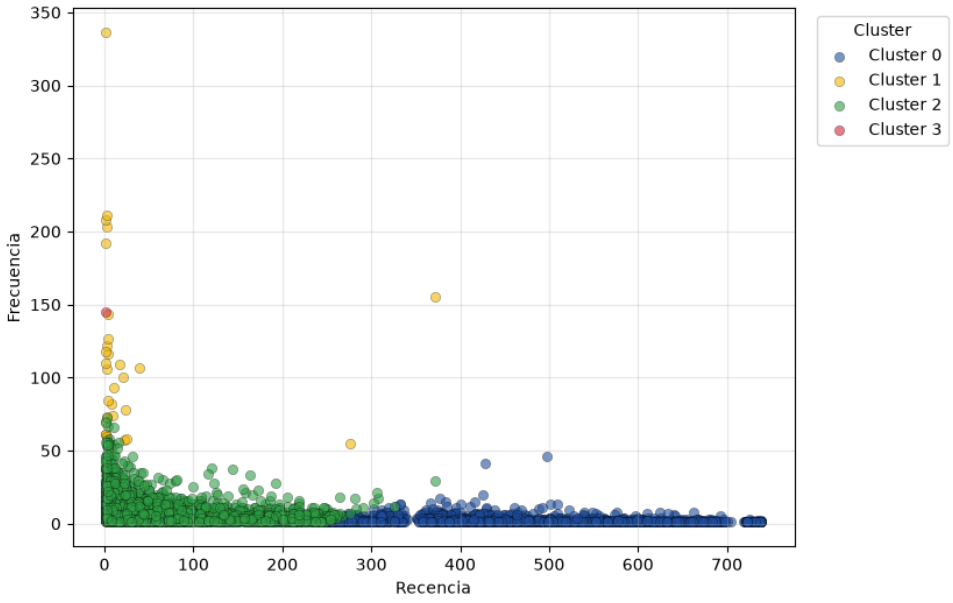


Figura 3.3: Visualización de los *clusters* ($k = 4$) en el plano Recencia-Frecuencia.

aquí se divide en dos. Por un lado queda un grupo chico de 21 clientes con un comportamiento de compra extremo, con 125 facturas y $103.602 de gasto promedio. Por otro lado queda un grupo más grande, de 315 clientes, que también compran seguido y gastan bastante (30 facturas y $14.647,99 en promedio), pero no tanto como el grupo anterior. Esto permite ver con más detalle a los clientes de mayor valor, algo que con $k = 4$ quedaba todo junto en un solo *cluster*. Al igual que con $k = 4$, aparece el *cluster* 4 que contiene solamente al mismo cliente con valores muy altos.

Con k=5, la separación entre los clientes queda un poco más fina. En la Figura 3.9 aparece un grupo nuevo que con $k = 4$ no se distinguía tanto: clientes con recencia baja o media y algo más de frecuencia que el resto, que forman una franja propia entre el grupo más inactivo y el más numeroso. Al igual que con $k = 4$, el cliente con el monto extremo vuelve a ocultar la diferencia entre los *clusters* en los gráficos que incluyen el monto, tanto para la Figura 3.10 y 3.12, así que se armaron las mismas versiones con zoom para este caso. En la Figura 3.11 se nota mucho mejor cómo quedan los tres grupos principales: uno con los montos más altos, otro en un nivel intermedio, y el más numeroso con los montos más bajos. Lo mismo se repite al mirar la Recencia en la Figura 3.13, donde esa separación se ve bastante más prolija que la que se había obtenido con $k = 4$.

3.3 Resultados de DBSCAN

Tabla 3.3: Resumen de *clusters*: DBSCAN (`eps`=0.2, `min_puntos`=5).

Cluster	Clientes	Recencia promedio	Frecuencia promedio	Monto promedio
Ruido (−1)	182	126,33	42,05	32.350,80
0	5.153	205,49	4,98	1.638,17
1	7	324,57	12,00	4.051,40
2	4	71,50	28,50	3.881,86
3	4	405,25	11,50	4.004,72

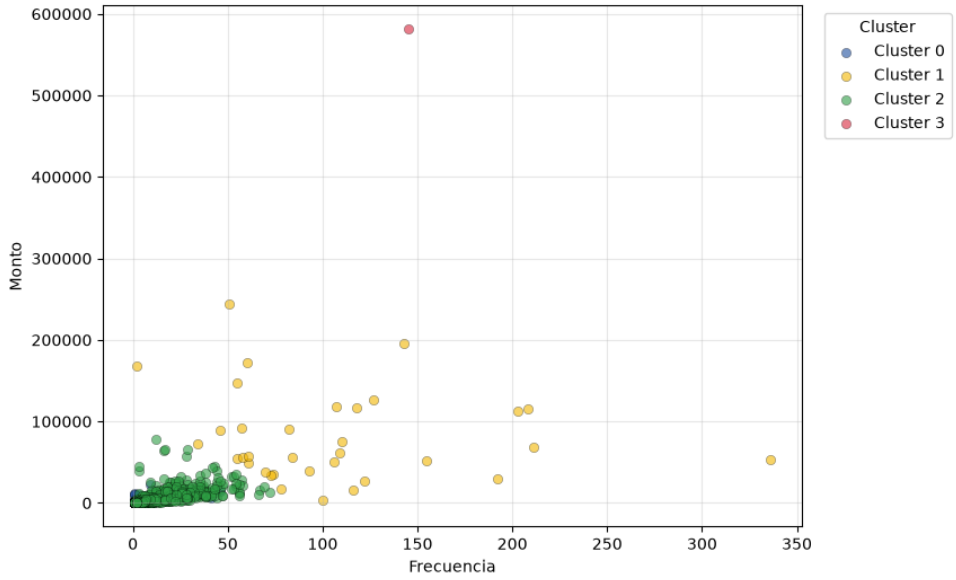


Figura 3.4: Visualización de los *clusters* ($k = 4$) en el plano Frecuencia-Monto.

Observando la Tabla 3.3, que muestra el resumen de los *clusters* que se obtuvo con estos parámetros incluyendo el grupo de ruido (etiqueta -1), y la Figura 3.14 se ve el problema: el *cluster* 0 se queda con 5.153 de los 5.350 clientes, casi todos los registros. Los *clusters* 1, 2 y 3 son grupos de 4 a 7 clientes solamente, así que no sirven mucho para segmentar. Lo más llamativo pasa en el grupo de ruido. Esos 182 clientes tienen en promedio 42 compras y $32.350,80 de monto, muy por encima del resto. O sea, DBSCAN no armó un *cluster* para los clientes de mayor valor, sino que los dejó afuera como ruido, porque no cumplían el criterio de densidad que fijan `eps` y `min_puntos`.

Esto muestra una limitación de DBSCAN con este tipo de datos: como usa un solo valor de `eps` para todo el *dataset*, no logra adaptarse tanto a la zona densa de clientes habituales como a los clientes con comportamiento extremo. Estos últimos terminan casi todos como ruido en vez de agruparse. Esta limitación es la que lleva a probar HDBSCAN en la siguiente sección.

Para DBSCAN se armaron los mismos gráficos de a pares de variables. Acá el resultado es bastante distinto al de *K-means*. Como se puede observar en la Figura 3.15 se ve que casi todos los clientes quedan metidos en un solo grupo, el *Cluster* 0, que se concentra en recencia baja y frecuencia baja. El resto de los puntos, los que tienen algún valor más alto o más raro en cualquiera de las variables, caen en el *Cluster* -1, que no es en realidad un grupo de clientes parecidos entre sí, sino los casos que el algoritmo no pudo asignar a ningún *cluster* (ruido). Esto se nota todavía más al mirar el Monto: en la Figura 3.16 los *clusters* 1, 2 y 3 son grupitos muy chicos, casi no se llegan a ver, y en la Figura 3.17 vuelve a aparecer el mismo cliente con el monto extremo, esta vez catalogado como ruido. En el fondo, lo que está haciendo DBSCAN acá no es separar a los clientes en distintos perfiles de comportamiento, sino más bien diferenciar entre los que se parecen a la mayoría y los que no. Esto confirma lo que ya se había visto al probar distintas combinaciones de `eps` y `min_puntos`: no hay ninguna configuración que logre armar grupos parejos y con sentido para el negocio, como sí se consigue con *K-means*.

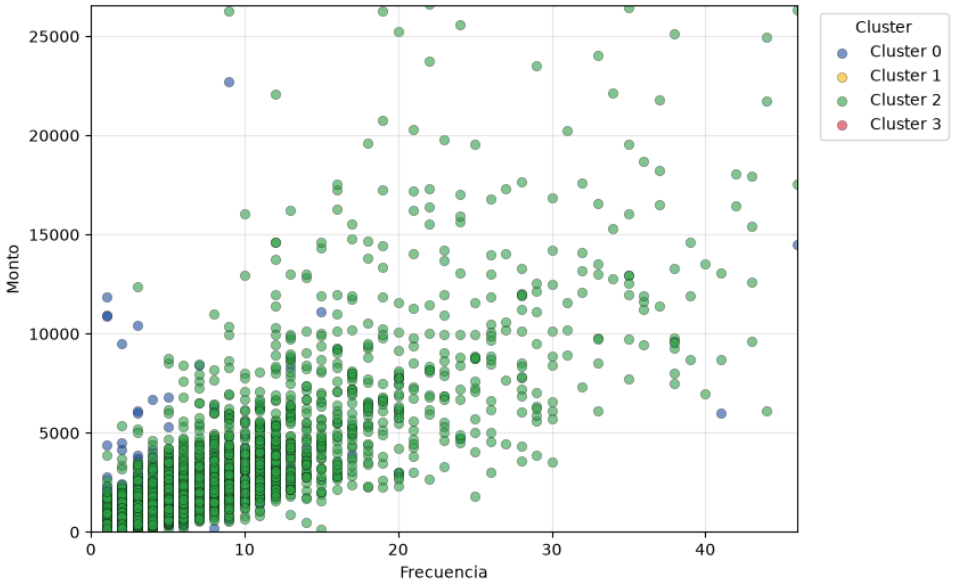


Figura 3.5: Visualización con zoom de los *clusters* ($k = 4$) en el plano Frecuencia-Monto.

3.4 Resultados de HDBSCAN

HDBSCAN con `min_cluster_size=500`, `min_samples=3`

Tabla 3.4: Resumen de *clusters*: HDBSCAN (`min_cluster_size`=500, `min_samples`=3).

Cluster	Clientes	Recencia promedio	Frecuencia promedio	Monto promedio
Ruido (−1)	1.760	227,15	13,00	6.401,98
0	550	48,21	7,20	2.176,17
1	1.146	122,09	3,74	1.072,27
2	520	108,38	2,00	599,32
3	627	133,36	1,00	300,04
4	747	508,52	1,00	262,74

A diferencia de DBSCAN, observando la Tabla 3.4 y la Figura **??** se puede ver que acá los *clusters* tienen tamaños mucho más parejos, entre 520 y 1.146 clientes cada uno, y se pueden diferenciar bastante bien entre sí. El *cluster* 0 agrupa a los clientes más activos: compraron hace poco (48 días en promedio) y con una frecuencia relativamente alta (7,2 compras). El *cluster* 4, en el otro extremo, junta a los clientes más inactivos: hace más de 500 días que no compran y solo hicieron una compra en total. Los *clusters* 2 y 3 son parecidos entre sí, ambos con clientes de baja frecuencia (1 a 2 compras) y montos bajos, pero se diferencian en la recencia. El grupo de ruido vuelve a ser el más interesante: son 1.760 clientes, casi un tercio de los registros totales, con un monto promedio de $6.401,98, muy por encima de cualquiera de los *clusters* formados. Esto confirma lo que se planteó en la metodología: HDBSCAN no fuerza a estos clientes de comportamiento más variado a entrar en algún grupo, sino que los deja afuera como puntos que no encajan claramente en ninguna densidad definida. Con HDBSCAN el resultado se ve distinto a lo que pasaba con DBSCAN. Acá el ruido (*Cluster* -1) no se concentra solamente en los casos más raros o extremos, sino que aparece repartido a lo largo de todo el rango de recencia, como se ve en la Figura 3.19 hay clientes marcados como ruido tanto entre los que compraron hace poco como entre los que hace mucho que no vuelven. Los cinco *clusters* que sí arma el algoritmo, en cambio, quedan todos agrupados

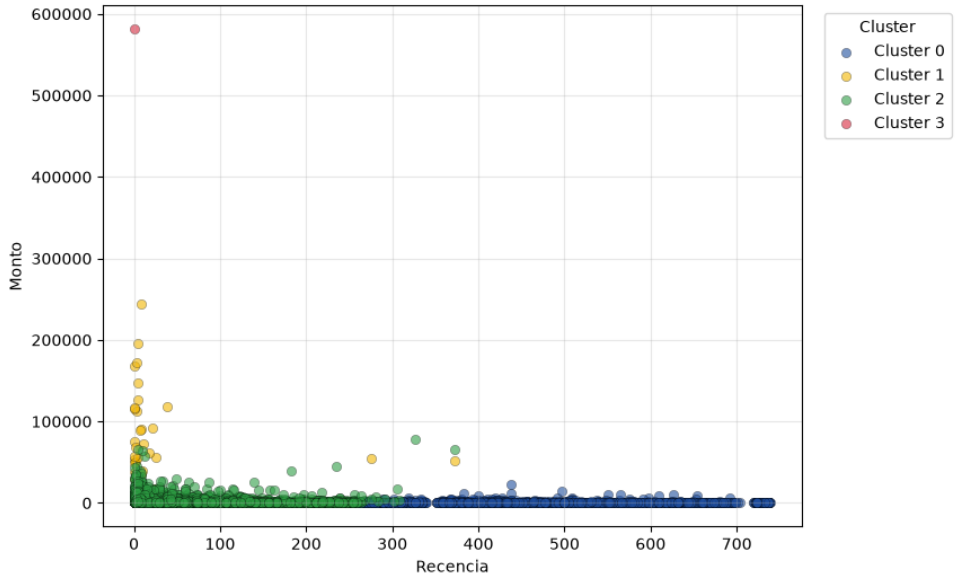


Figura 3.6: Visualización de los *clusters* ($k = 4$) en el plano Recencia-Monto.

en frecuencia y monto bajos, bastante pegados entre sí, algo que se repite tanto en la Figura 3.20 como en la Figura 3.22. Por eso, al igual que se hizo con *K-means*, se armó una versión con zoom de estos dos gráficos, dejando afuera de la vista al cliente con el monto más alto. En la Figura 3.21 se ve que los cinco *clusters* no están mezclados entre sí, sino que se ordenan en franjas bastante marcadas, cada uno con un rango distinto de frecuencia y monto, quedando casi como escalones desde los clientes con menor actividad de compra hasta los que más compran y más gastan dentro de este grupo. Lo mismo se aprecia mirando la Recencia en la Figura 3.23, donde esa misma separación en franjas se mantiene.

La Figura 3.24 muestra el árbol condensado que construye HDBSCAN a partir del árbol de expansión mínima. En esta figura, el eje vertical representa el valor de λ (inversamente proporcional a la distancia), y el ancho de cada rama indica la cantidad de puntos que la componen. Se puede ver cómo, a medida que aumenta λ es decir, a medida que se pide más densidad, todos los clientes se van separando en ramas cada vez más finas. Al principio del gráfico se desprende una rama ancha que junta a la mayoría de los clientes, y más abajo esa rama se divide en varias más chicas que llegan hasta el final del árbol. Esas son las que HDBSCAN termina eligiendo como *clusters* finales. Las ramas que se cortan antes de llegar al final son agrupaciones que el algoritmo no consideró lo bastante estables como para ser un *cluster*, y esos puntos terminan como ruido.

HDBSCAN con $min_cluster_size = 200, min_samples = 1$

Con esta configuración, los nueve *clusters* muestran una especie de gradiente de valor de cliente, como puede verse en la Tabla 3.5 y en la Figura 3.18. En un extremo están el *Cluster* 0 y el *Cluster* 1, con la recencia más baja y la frecuencia más alta del grupo, son los clientes que compran seguido y hace poco, algo que se aprecia bien en la Figura 3.26. En el otro extremo aparecen el *Cluster* 2 y el *Cluster* 8, con recencias arriba de los 400 días y apenas una o dos compras en total, que representan a los clientes prácticamente perdidos. En el medio se ubican el resto de los *clusters*, con distintas combinaciones de recencia y frecuencia que van marcando ese pasaje de un extremo al otro. Los *clusters* 3, 4, 5 y 6 completan ese pasaje entre los dos extremos, y aunque a simple vista parecen parecidos entre sí, tienen matices que los distinguen. El *Cluster* 4 y el *Cluster* 5 tienen la recencia más baja de este grupo intermedio (86 y 83 días) junto con una frecuencia moderada (5 y 4 compras), lo que los deja como clientes activos aunque no tan seguido como los del *Cluster* 0 o el *Cluster* 1. El *Cluster* 3, en cambio, tiene una recencia bastante más alta (108 días) pero solo 2 compras en promedio, mientras que el *Cluster* 6 es el que más se acerca

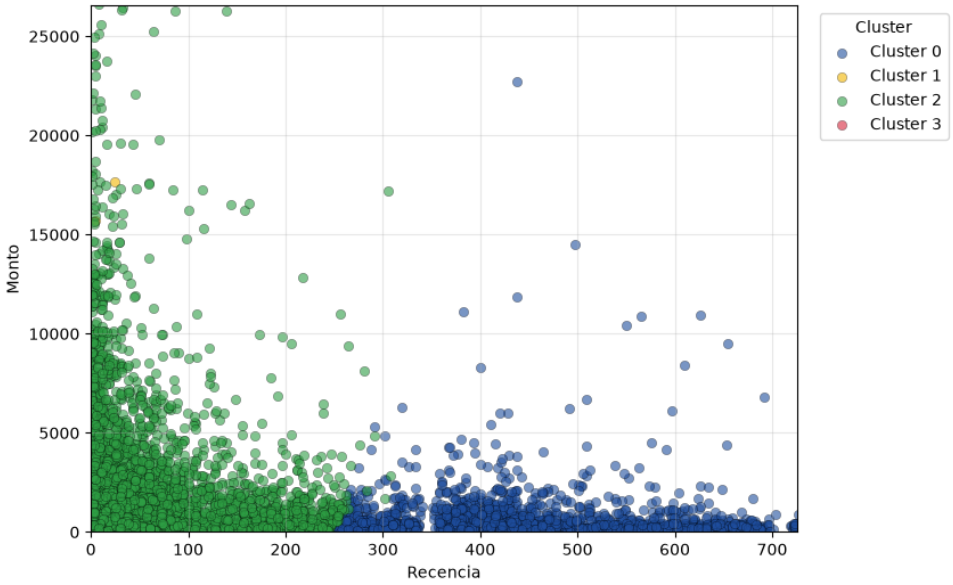


Figura 3.7: Visualización con zoom de los *clusters* ($k = 4$) en el plano Recencia-Monto.

Cluster	Cantidad de clientes	Recencia promedio	Frecuencia promedio	Monto promedio
-1	1.173	203,25	16,69	8.700,18
0	218	30,29	10,96	3.299,60
1	575	49,19	7,22	2.249,39
2	313	467,77	2,00	533,26
3	520	108,38	2,00	599,32
4	277	86,28	5,00	1.511,66
5	317	83,15	4,00	1.126.,10
6	575	164,57	3,00	902,58
7	633	132,91	1,00	313,56
8	749	508,62	1,00	266,35

Tabla 3.5: Perfil de los *clusters* obtenidos con HDBSCAN (`min_cluster_size`=200, `min_samples`=1).

al extremo de clientes perdidos dentro de este grupo intermedio, con una recencia de 164 días y apenas 3 compras. En conjunto, estos cuatro *clusters* muestran que entre çliente activoz çliente perdido"no hay un corte tajante, sino una transición bastante gradual. Vale la pena remarcar el caso del *Cluster* -1: a diferencia de lo que suele pasar, acá el ruido no representa a los clientes menos valiosos, sino todo lo contrario: tiene el monto promedio más alto de todos los grupos ($8.700 aproximadamente), muy por encima de cualquiera de los nueve *clusters*, tal como se ve en las Figuras 3.27 y 3.29. Esto tiene sentido porque HDBSCAN termina descartando como ruido a los casos que no encajan bien en ningún patrón, y entre esos casos atípicos hay varios clientes de compra muy grande que no logran agruparse entre sí por ser pocos y no tener un comportamiento parecido. En cuanto a los gráficos, se armó una versión con zoom de los que incluyen el Monto, dejando afuera de la vista al cliente con el monto más alto. En la Figura 3.28 se ve que los nueve *clusters* se ordenan en franjas bastante marcadas según su frecuencia y monto. Esto se aprecia todavía mejor en la Figura 3.30, donde se nota algo particular: el *Cluster* 7 y el *Cluster* 8 agrupan a clientes que compraron una sola vez, pero quedan bien separados por la recencia: el *Cluster* 7 son clientes que compraron hace relativamente poco (recencia promedio de 133 días), mientras que el *Cluster* 8 son clientes que probaron una vez y nunca más volvieron (recencia promedio de casi 509

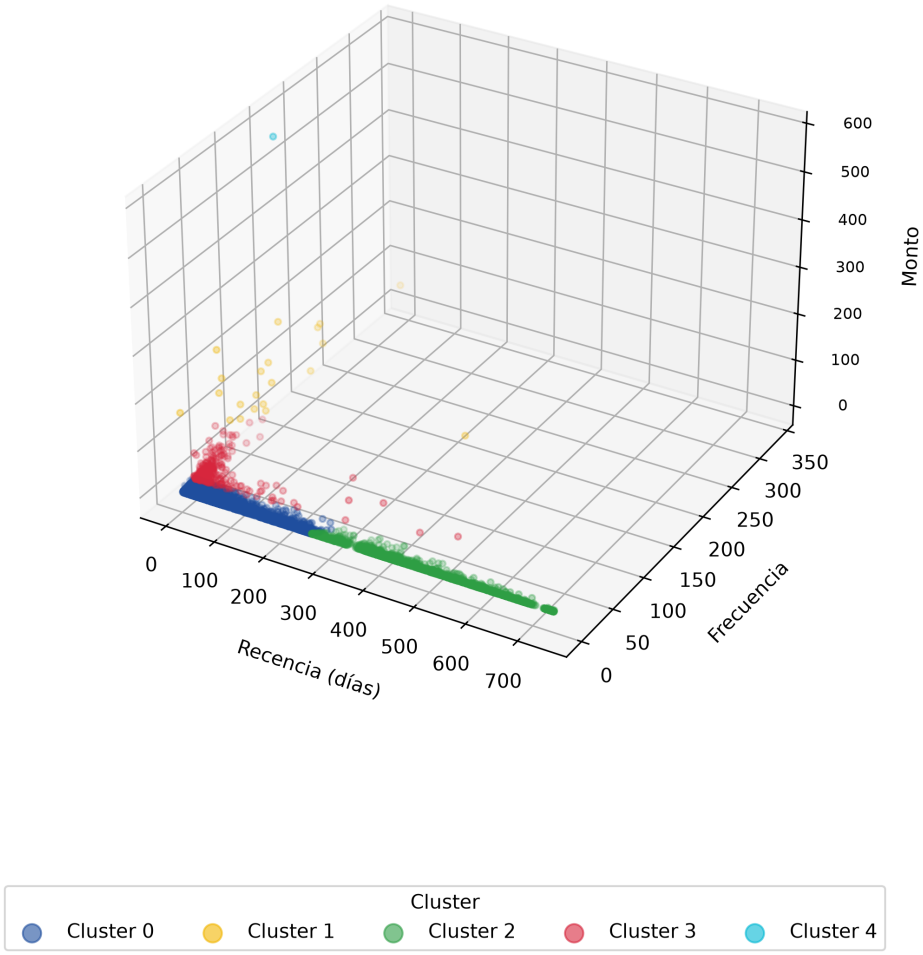


Figura 3.8: Distribución de clientes según *K-Means*: $k = 5$.

días). Es una distinción que con menos *clusters* no se llegaba a ver, y que tiene bastante sentido para pensar estrategias de recontacto distintas para cada uno de estos dos grupos.

3.5 Evaluación de los modelos

La Tabla 3.6 muestra los resultados de las tres métricas utilizadas para comparar los distintos modelos: Silhouette, Davies-Bouldin y Calinski-Harabasz. *K-means* con $k = 5$ es el que mejor puntúa en las tres, seguido por *K-means* con $k = 4$. DBSCAN queda muy por detrás en todos los casos, lo cual confirma la limitación que ya se había visto en los gráficos: al concentrar casi todos los clientes en un único *cluster*, no logra una segmentación real. Un resultado que vale la pena remarcar es que, entre las dos configuraciones de HDBSCAN, la que mejor puntúa en las métricas es la primera (`min_cluster_size`=500, `min_samples`=3), a pesar de que se había probado también la segunda configuración (`min_cluster_size`=200, `min_samples`=1) por dar una segmentación con más sentido para el negocio. Esto tiene que ver con que estas métricas premian *clusters* compactos y bien separados entre sí. La configuración de 9 *clusters*, en cambio, encuentra matices más finos entre los clientes, como la distinción entre compradores únicos recientes y perdidos, que valen la pena desde el punto de vista del negocio, aunque las métricas internas no lo reflejen tan bien. En conjunto, *K-means* con $k = 5$ se queda como el modelo con mejor desempeño según estas métricas, lo cual coincide con que fue el más fácil de

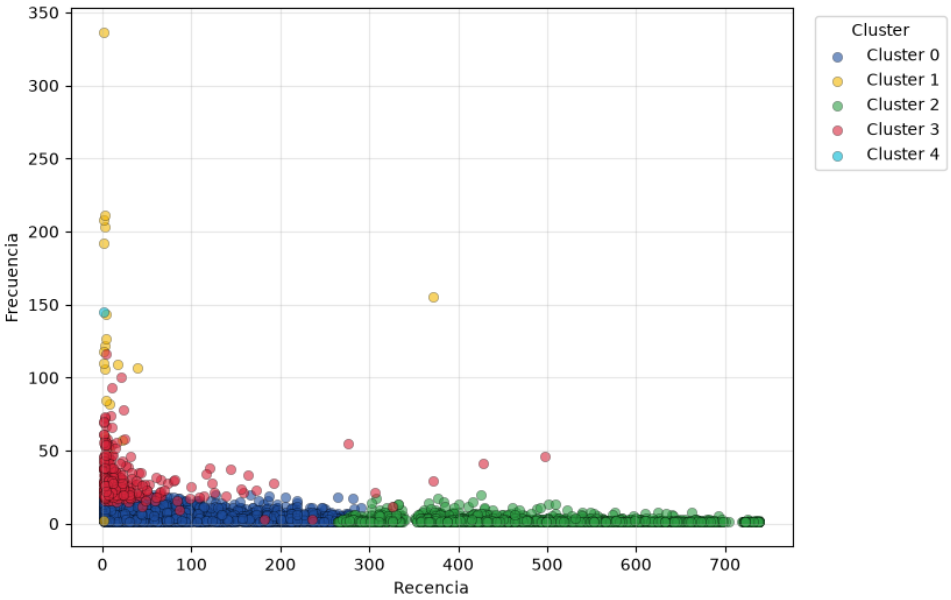


Figura 3.9: Visualización de los *clusters* ($k = 5$) en el plano Recencia-Frecuencia.

Modelo	Silhouette	Davies-Bouldin	Calinski-Harabasz
K-Means ($k = 4$)	0,5757	0,5073	3.842,37
K-Means ($k = 5$)	0,6000	0,5653	4.581,65
DBSCAN	-0,0368	1,0570	7,13
HDBSCAN (500/3)	0,1412	3,8249	2.219,82
HDBSCAN (200/1)	-0,0290	4,6234	1.686,66

Tabla 3.6: Comparación de métricas de evaluación entre los distintos modelos.

interpretar y el que dio una segmentación más pareja a lo largo de todo el análisis.

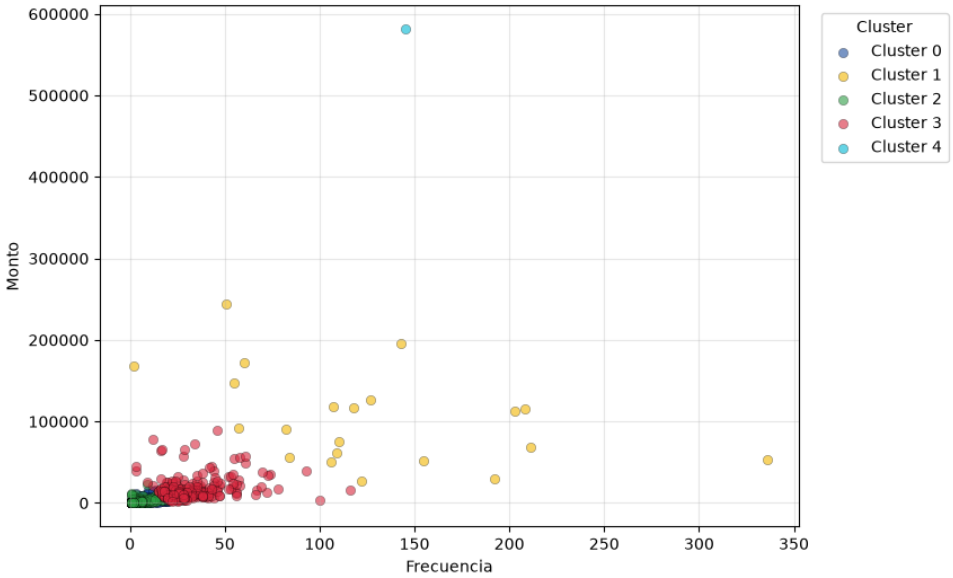


Figura 3.10: Visualización de los *clusters* ($k = 5$) en el plano Frecuencia-Monto.

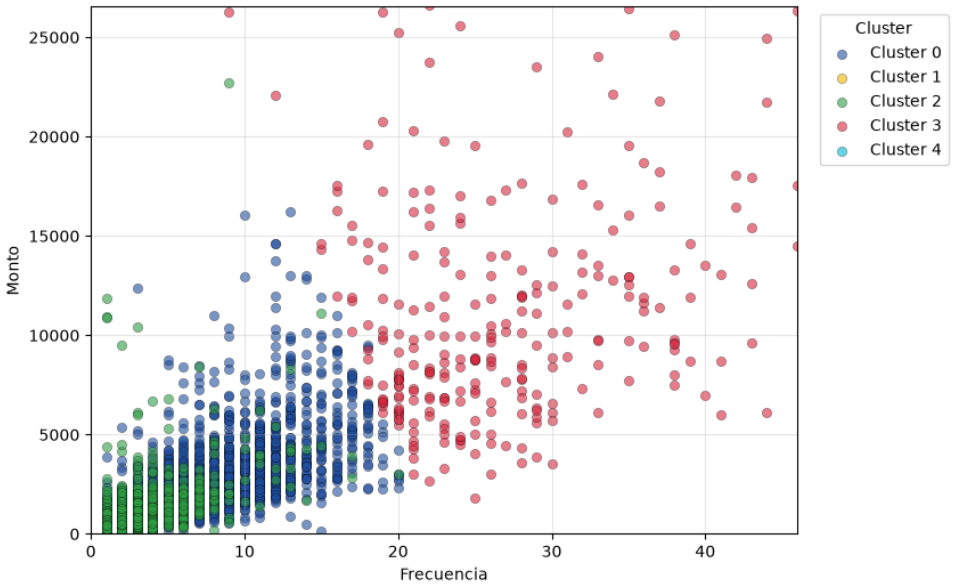


Figura 3.11: Visualización con zoom de los *clusters* ($k = 5$) en el plano Frecuencia-Monto.

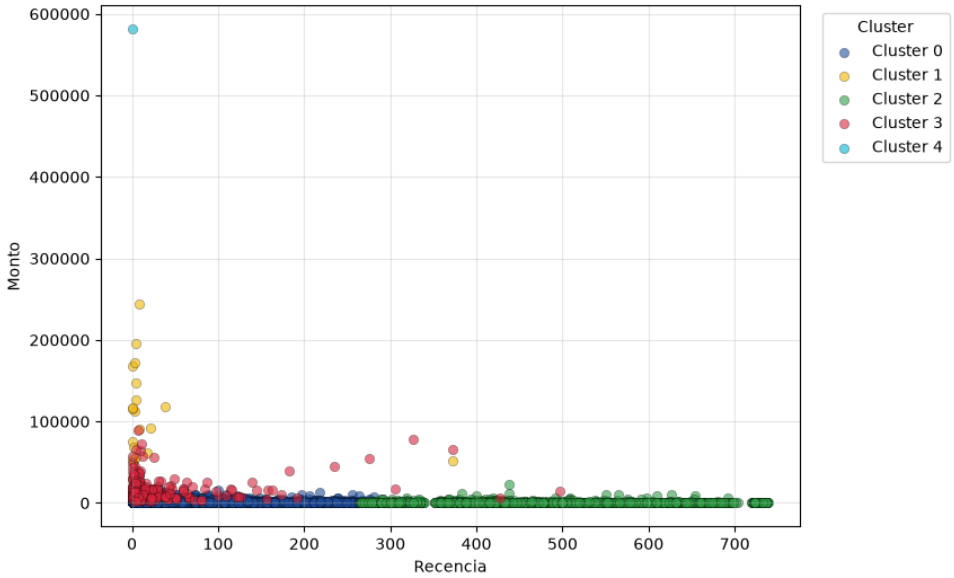


Figura 3.12: Visualización de los *clusters* ($k = 5$) en el plano Recencia-Monto.

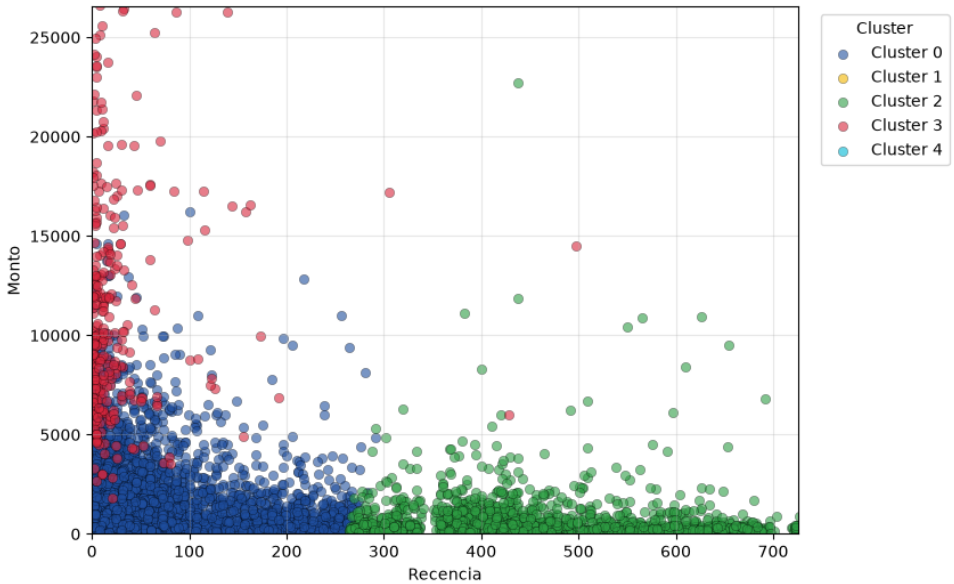


Figura 3.13: Visualización con zoom de los *clusters* ($k = 5$) en el plano Recencia-Monto.

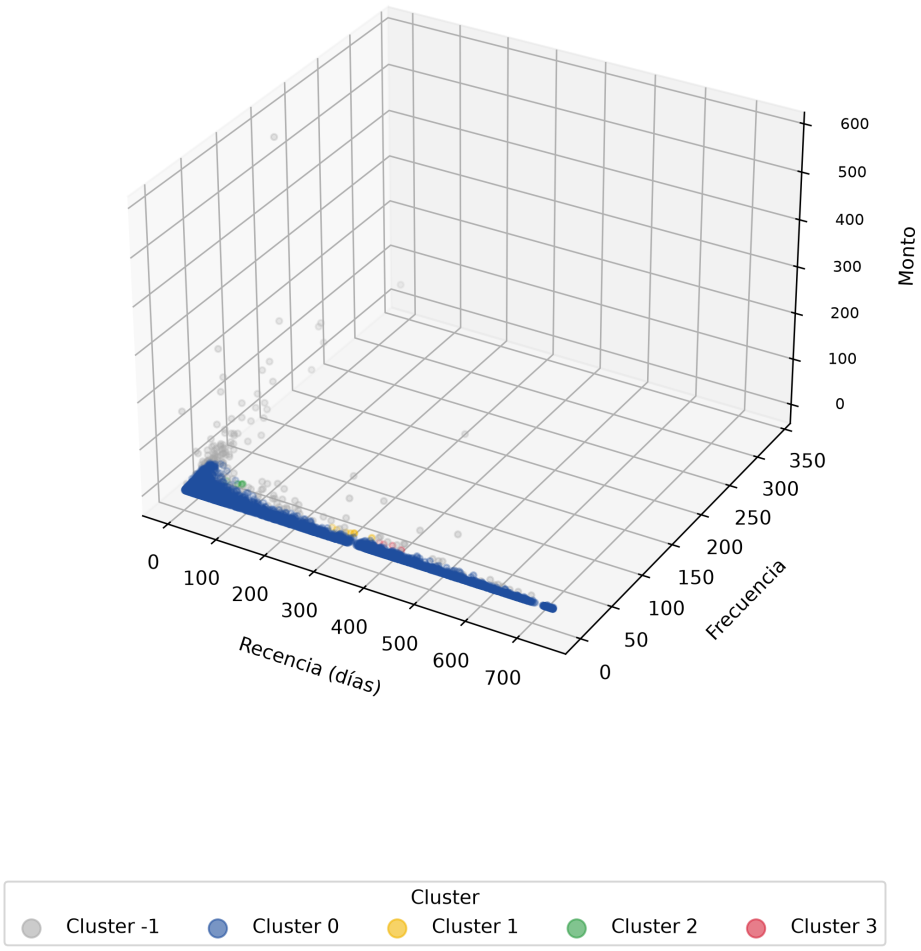


Figura 3.14: Distribución de clientes según DBSCAN.

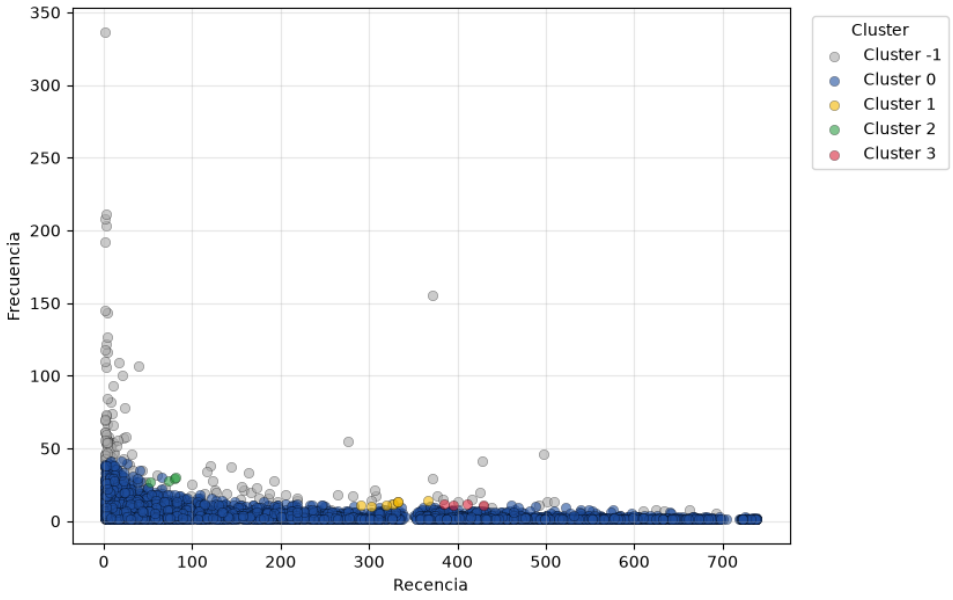


Figura 3.15: Visualización de los *clusters* DBSCAN en el plano Recencia-Frecuencia.

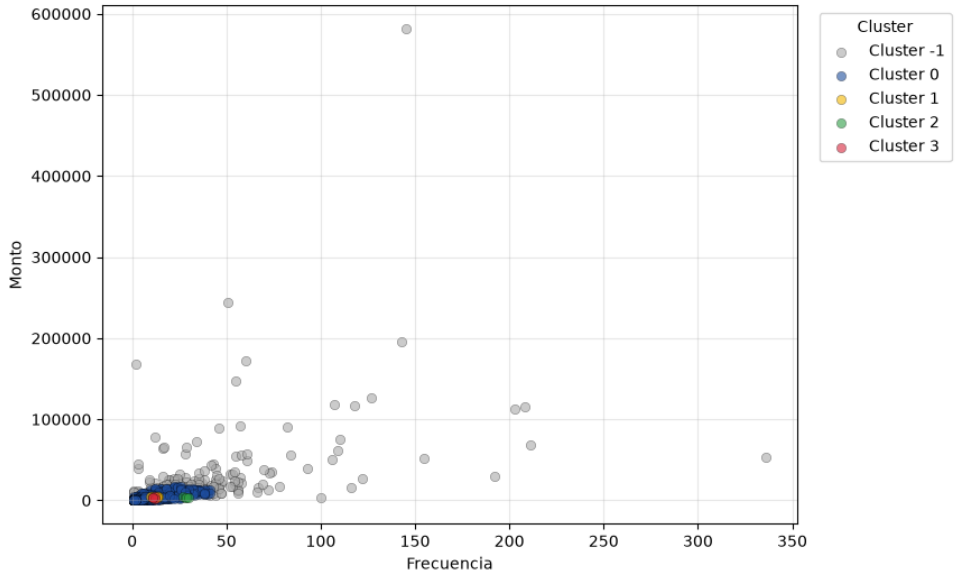


Figura 3.16: Visualización de los *clusters* DBSCAN en el plano Frecuencia-Monto.

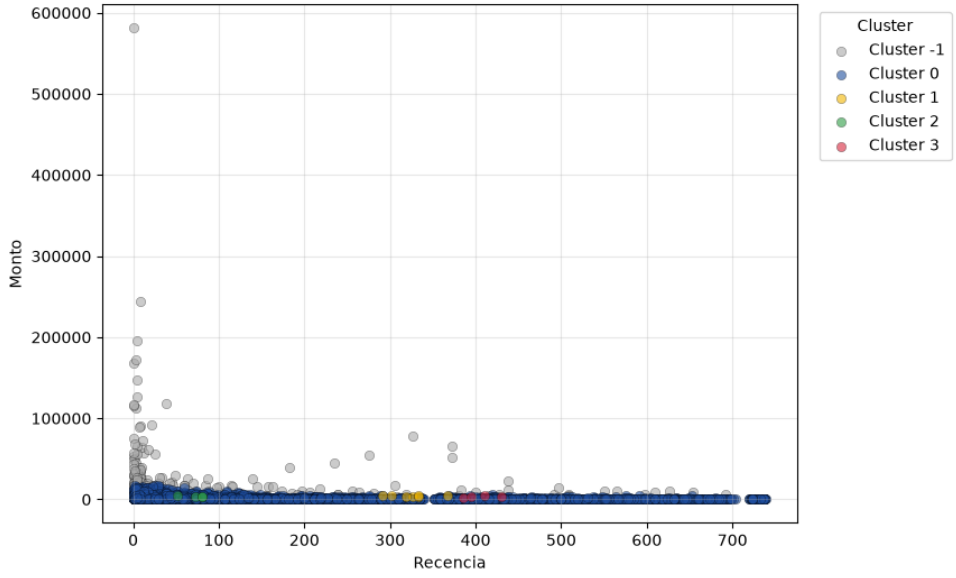


Figura 3.17: Visualización de los *clusters* DBSCAN en el plano Recencia-Monto.

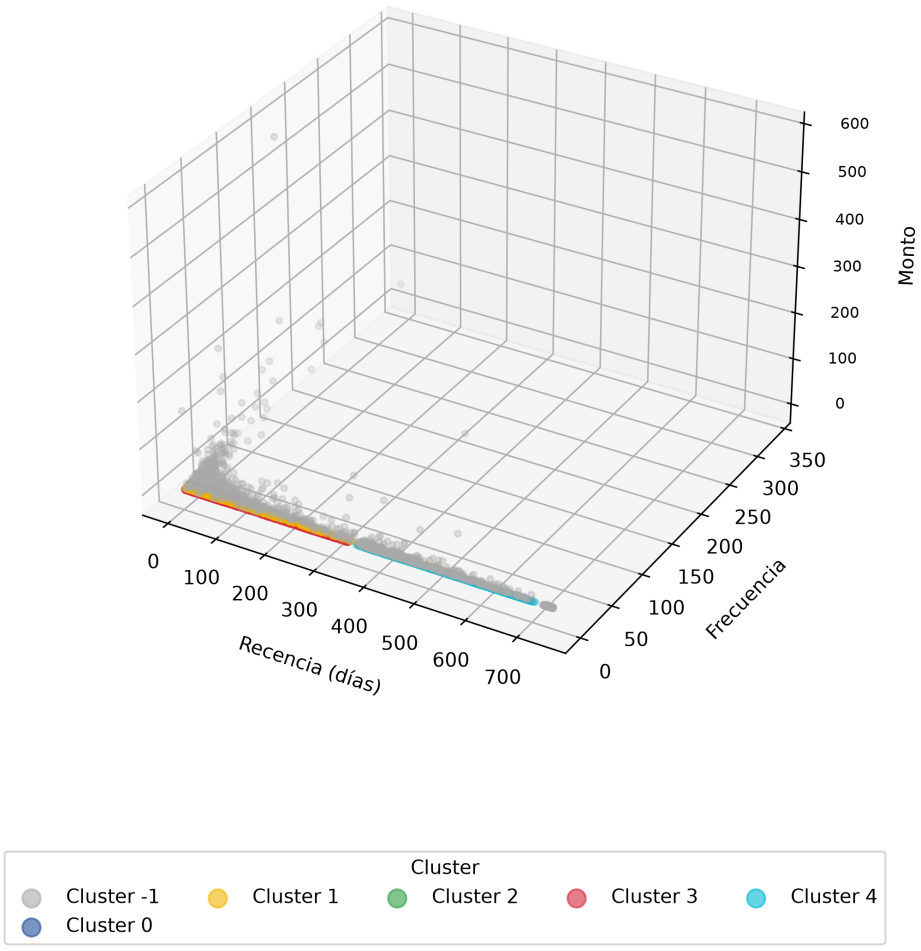


Figura 3.18: Distribución de clientes según HDBSCAN - `min_cluster_size`=500, `min_samples`=3.

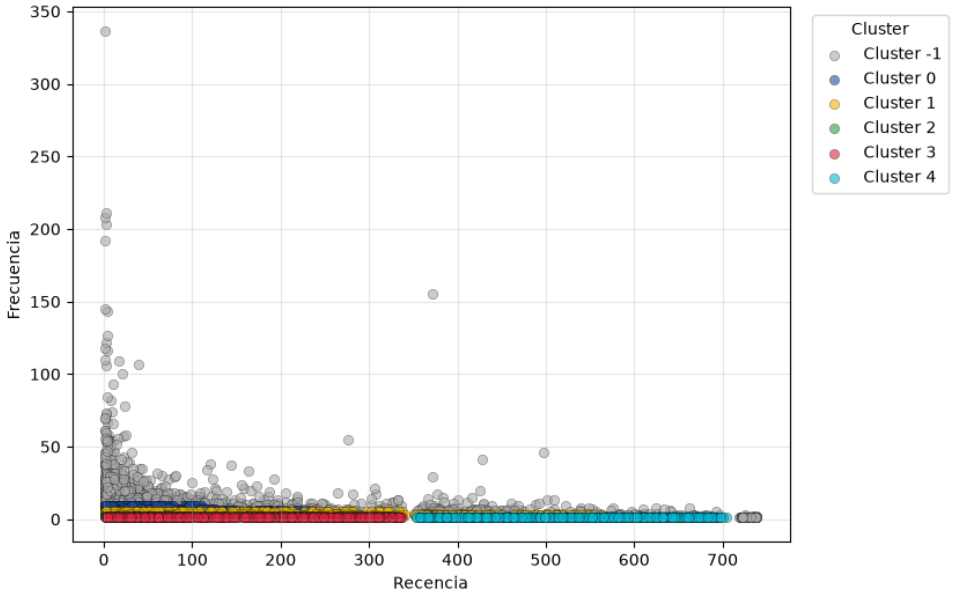


Figura 3.19: Visualización de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Recencia-Frecuencia.

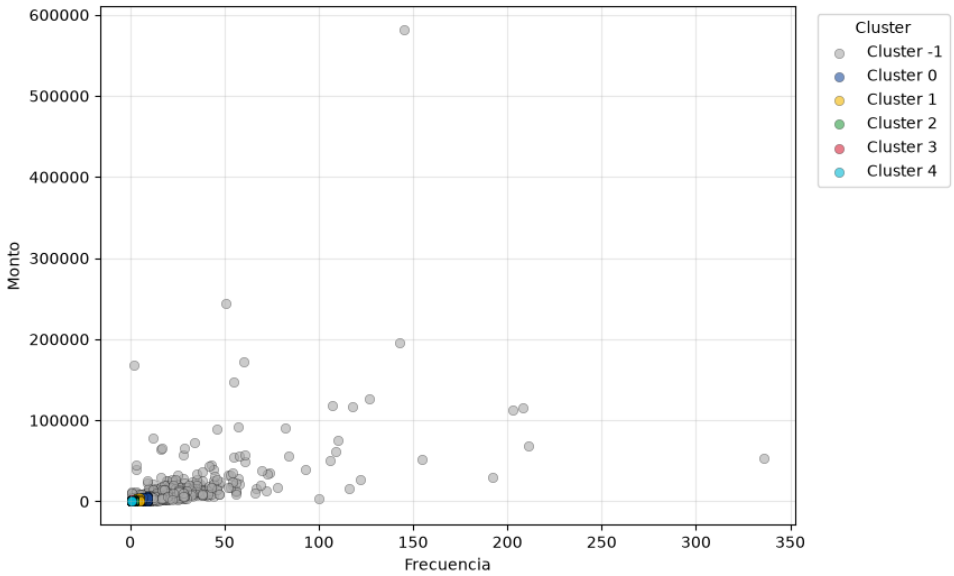


Figura 3.20: Visualización de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Frecuencia-Monto.

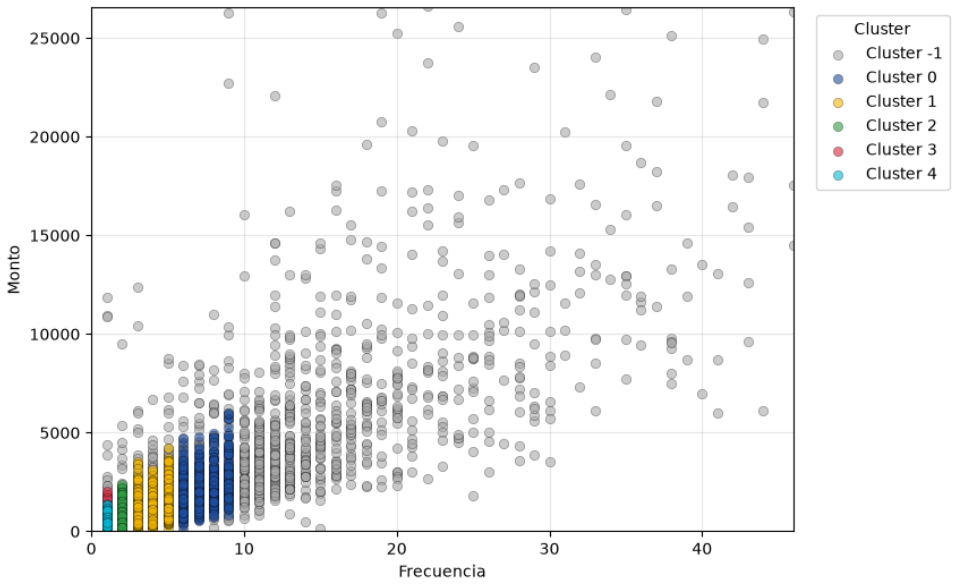


Figura 3.21: Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Frecuencia-Monto.

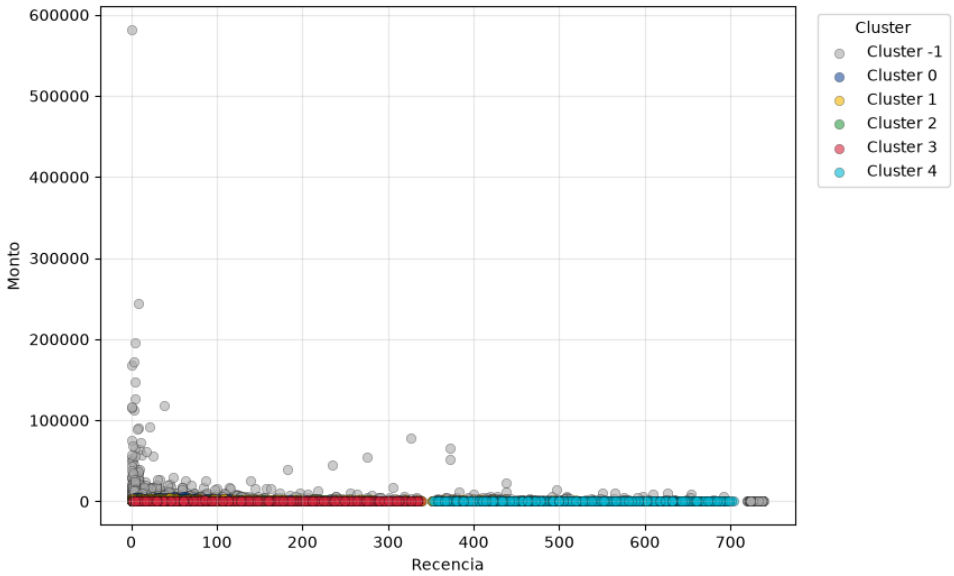


Figura 3.22: Visualización de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plaon Recencia-Monto.

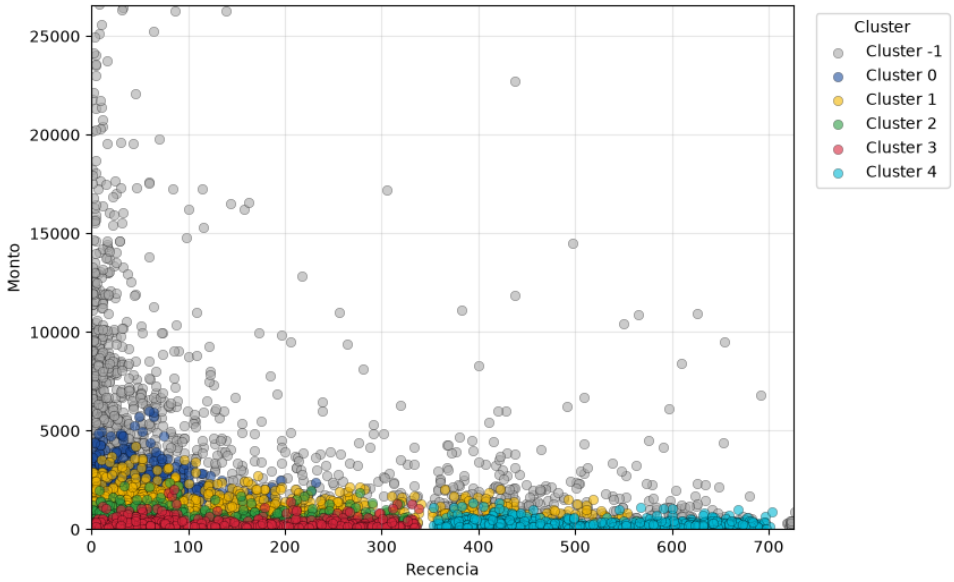


Figura 3.23: Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=500, `min_samples`=3) en el plano Recencia-Monto.

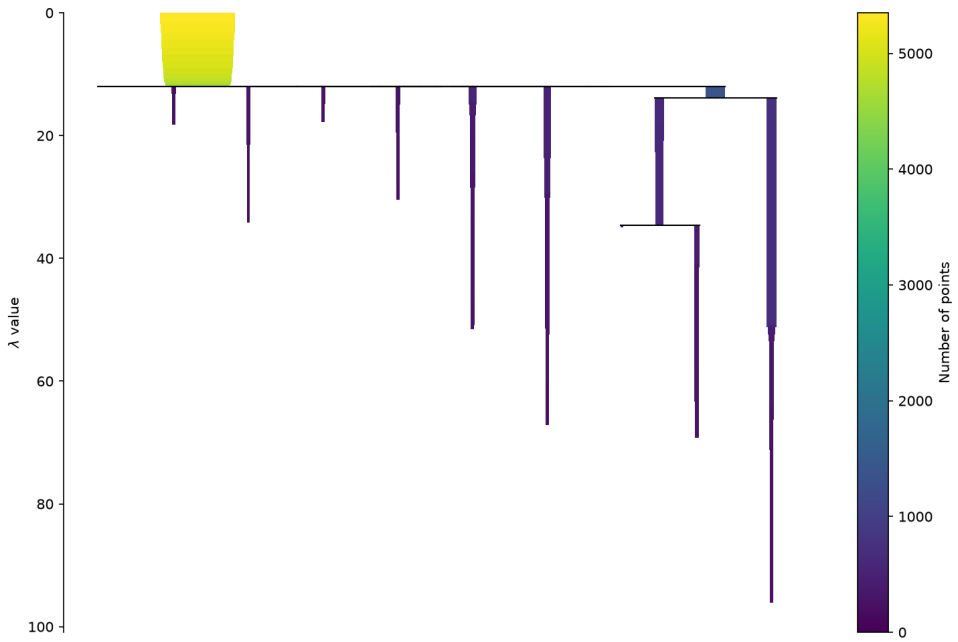


Figura 3.24: Jerarquía de *clusters* construida por HDBSCAN (`min_cluster_size`=500, `min_samples`=3).

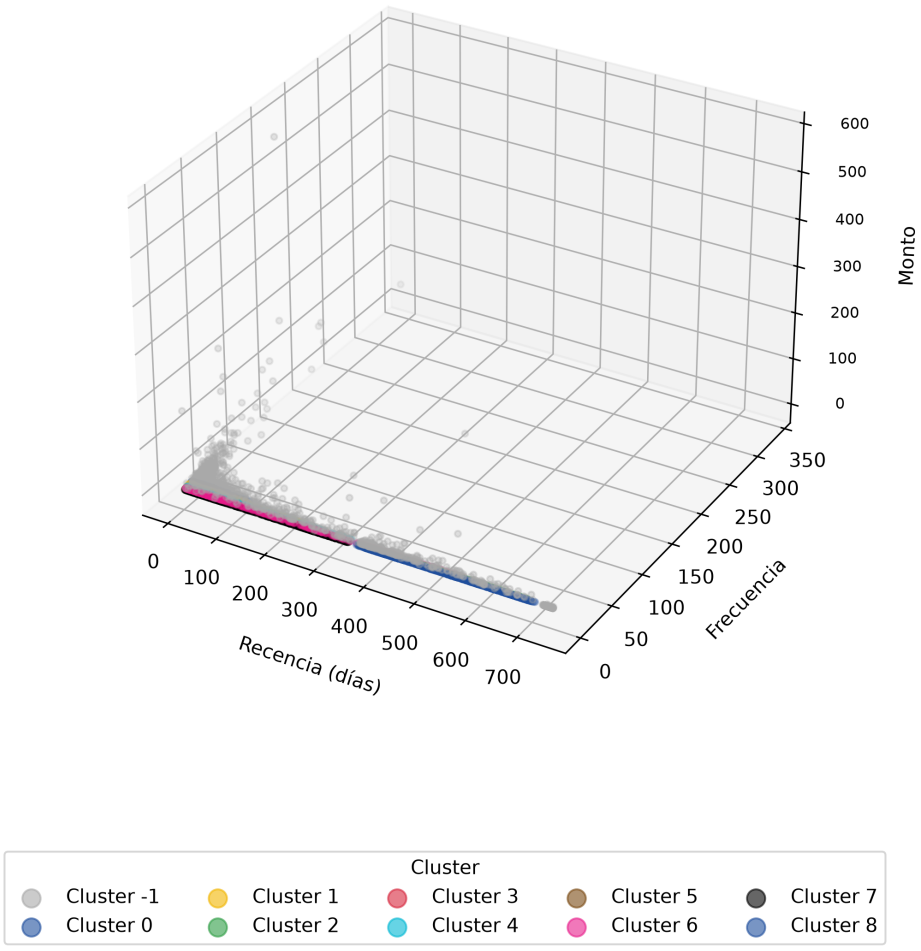


Figura 3.25: Distribución de clientes según HDBSCAN - `min_cluster_size`=200, `min_samples`=1 .

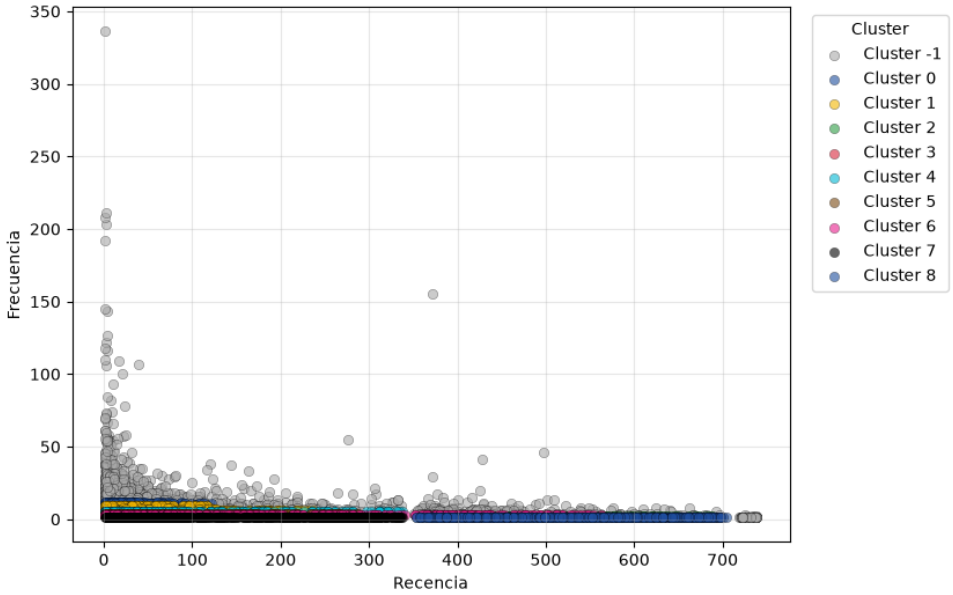


Figura 3.26: Visualización de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Recencia-Frecuencia.

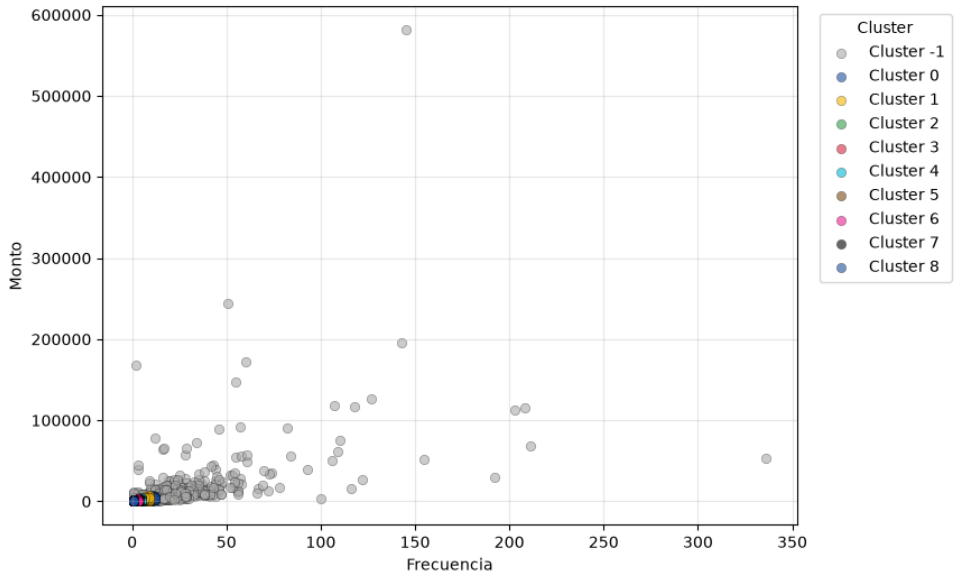


Figura 3.27: Visualización de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Frecuencia-Monto.

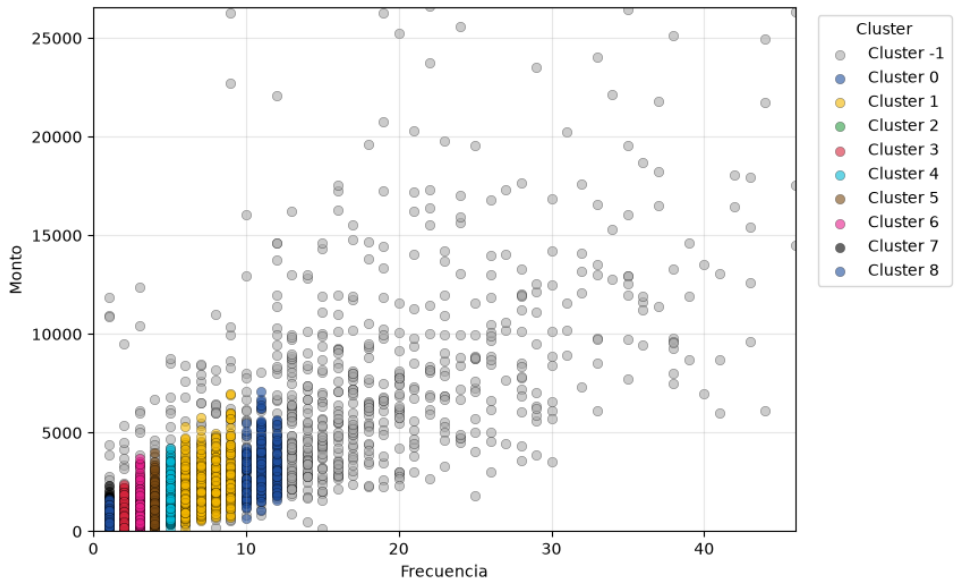


Figura 3.28: Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Frecuencia-Monto.

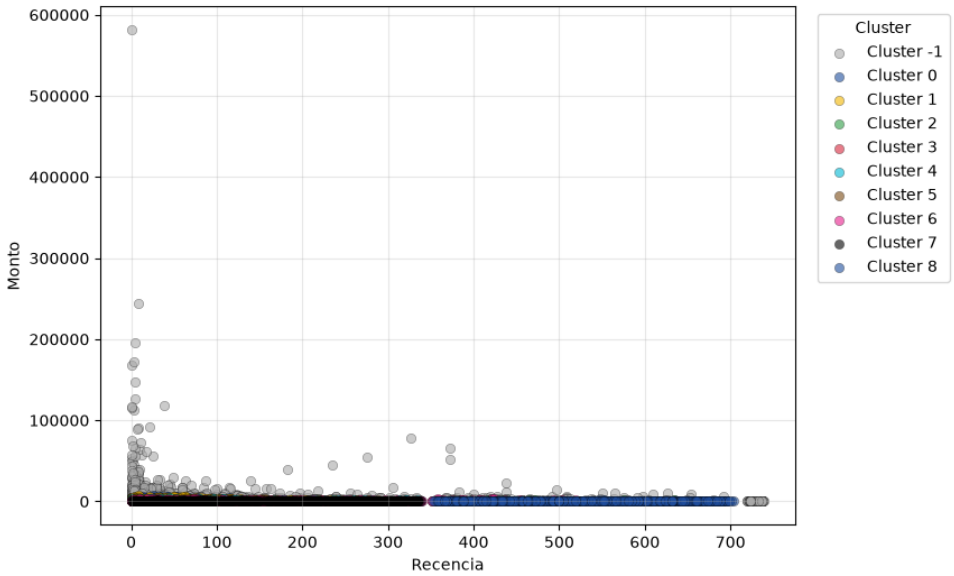


Figura 3.29: Visualización de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Recencia-Monto.

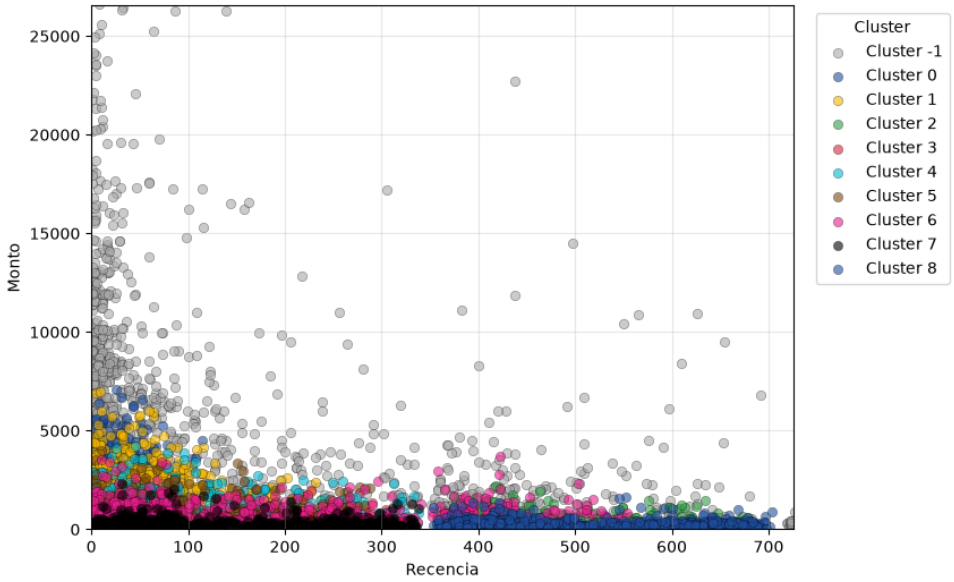


Figura 3.30: Visualización con zoom de los *clusters* HDBSCAN (`min_cluster_size`=200, `min_samples`=1) en el plano Recencia-Monto.

Capítulo 4

Conclusiones

En este trabajo se analizó y comparó el desempeño de los algoritmos de *clustering K-means*, DBSCAN e HDBSCAN combinados con el modelo RFM (Recencia, Frecuencia, Monto) para la segmentación de clientes de un comercio minorista en línea (*Online Retail II*).

A lo largo del desarrollo se observaron diferencias significativas en la forma en que cada método procesa la información y estructura los grupos, arribando a las siguientes conclusiones principales:

- **Desempeño de *K-means* en métricas de evaluación interna:** El algoritmo *K-means* con $k = 5$ presentó los mejores indicadores cuantitativos entre los modelos evaluados, alcanzando un coeficiente de Silhouette de 0,6000, un índice de Davies-Bouldin de 0,5653 y un valor de Calinski-Harabasz de 4.581,65. Estas métricas reflejan que *K-means* logró construir grupos bien delimitados y internamente compactos, delimitando de manera limpia a los clientes inactivos, los compradores habituales y las distintas categorías de mayor gasto.

- **Dificultades de DBSCAN ante datos con densidad variable:** La configuración probada de DBSCAN ($\epsilon = 0{,}2$, `min_puntos`=5) mostró limitaciones para generar una segmentación equilibrada. El algoritmo concentró al 96,3 % de la base de usuarios en un solo *clúster* general y marcó como ruido a casi la totalidad de los clientes con mayor frecuencia y gasto. Este resultado confirma en la práctica la dificultad de DBSCAN para responder a conjuntos de datos donde la densidad varía fuertemente entre la masa principal de usuarios y los compradores de mayor valor.

- **Relación entre métricas matemáticas y utilidad comercial en HDBSCAN:** Aunque HDBSCAN registró puntajes más bajos en los índices de evaluación sintéticos debido a la forma en que estas métricas penalizan la presencia de puntos de ruido o fronteras irregulares, la variante con `min_cluster_size=200` y `min_samples=11` aportó una perspectiva muy útil desde el punto de vista del análisis de negocio. Esta configuración permitió descomponer la base en 9 gradientes que reflejan con detalle las etapas intermedias entre un cliente activo y uno perdido, diferenciando, por ejemplo, a los compradores ocasionales recientes de aquellos que no registran actividad desde hace tiempo.

- **Gestión de valores extremos y clientes corporativos:** Un aspecto relevante del enfoque por densidad en HDBSCAN fue su tratamiento de las observaciones atípicas. Mientras que en *K-means* la presencia de un cliente con un volumen de compra excepcional ($580.987,04) genera un *clúster* de un único integrante y condiciona la escala del resto, HDBSCAN clasifica a estos usuarios con perfiles de compra extremos dentro de la categoría de ruido junto a otros clientes de facturación elevada (con un promedio cercano a los $8.700). De este modo, se evita distorsionar la agrupación de los clientes de consumo regular.

A partir de los resultados obtenidos, se concluye que no existe un único algoritmo superior en todos los aspectos, sino que sus aportes resultan complementarios:

- Para la gestión operativa y la automatización de campañas de marketing general, *K-means* (con $k = 5$) se presenta como la opción más adecuada por su simplicidad, rápido cómputo y consistencia en los indicadores de separación.
- Por su parte, HDBSCAN constituye una herramienta analítica valiosa para explorar la estructura profunda de los datos, aislar automáticamente cuentas corporativas o mayoristas y diseñar estrategias específicas de recontacto orientadas a subgrupos con hábitos de compra particulares.

Bibliografía